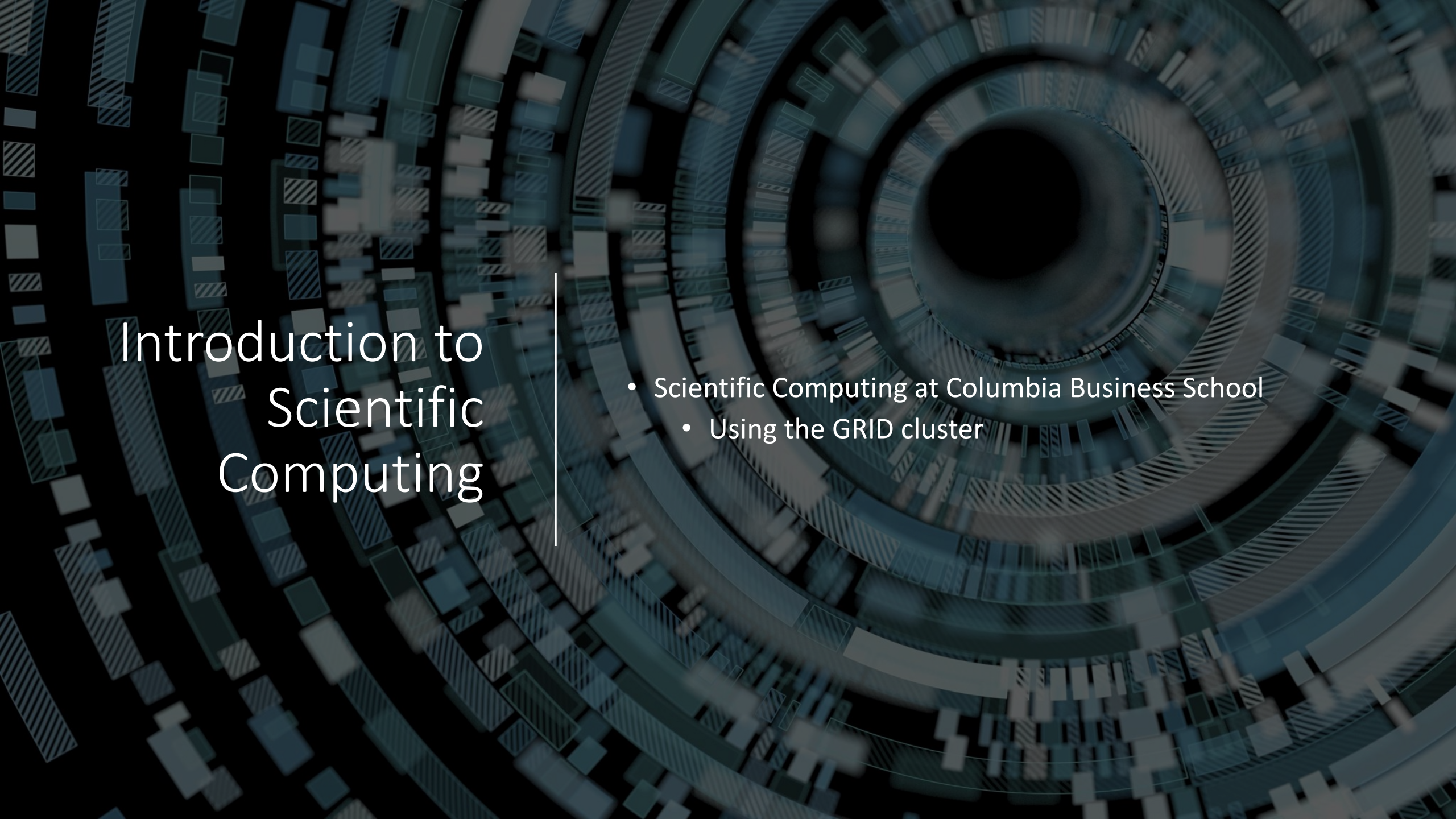




Research Computing @ CBS

Razvan N. Popescu
Research Computing Director

The background of the slide is a dark, abstract composition. It features a series of concentric circles that create a tunnel-like effect, drawing the eye towards the center. Overlaid on these circles are various digital patterns, including squares, rectangles, and lines, some of which are filled with a fine grid or cross-hatch texture. The overall color palette is a range of blues and greys, from deep navy to light, almost white, highlights.

Introduction to Scientific Computing

- Scientific Computing at Columbia Business School
 - Using the GRID cluster



Scientific computing

From IBM mainframes to minicomputers
and supercomputers...



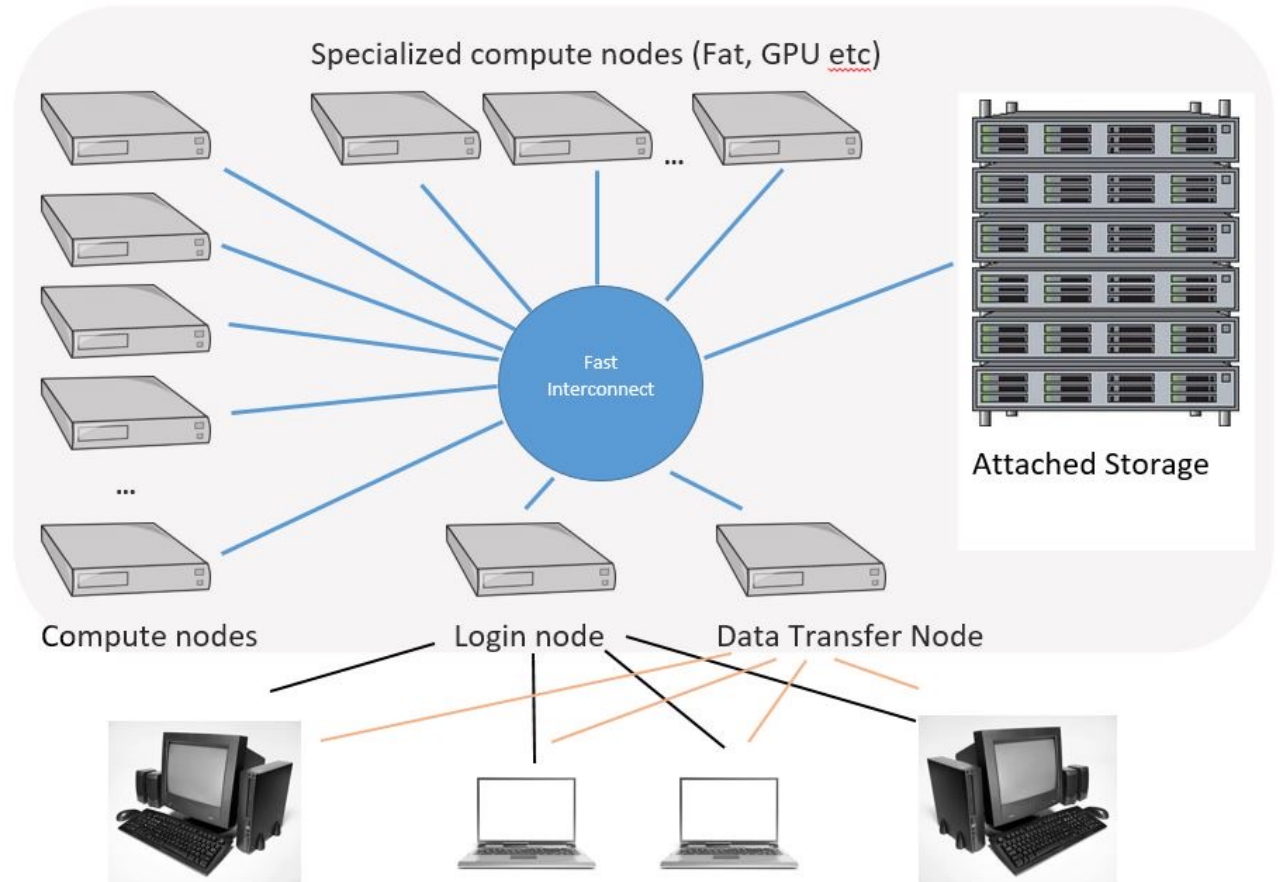
Scientific computing

To HPC clusters, the cloud

... and the GRID@CBS

Typical (simple) HPC cluster architecture

- Pool of compute nodes (homogeneous or heterogeneous)
- Specialized compute nodes (high mem, GPU, FPGA, etc.)
- Shared (high-performance) storage
- Interface nodes:
 - Login/Interactive node(s)
 - Data transfer node(s)
 - (*Admin node*)
- (*Infrastructure services*):
 - (**Scheduler**)
 - (*Backup*)
 - (*Identity management*)
 - (*Logging and monitoring*)

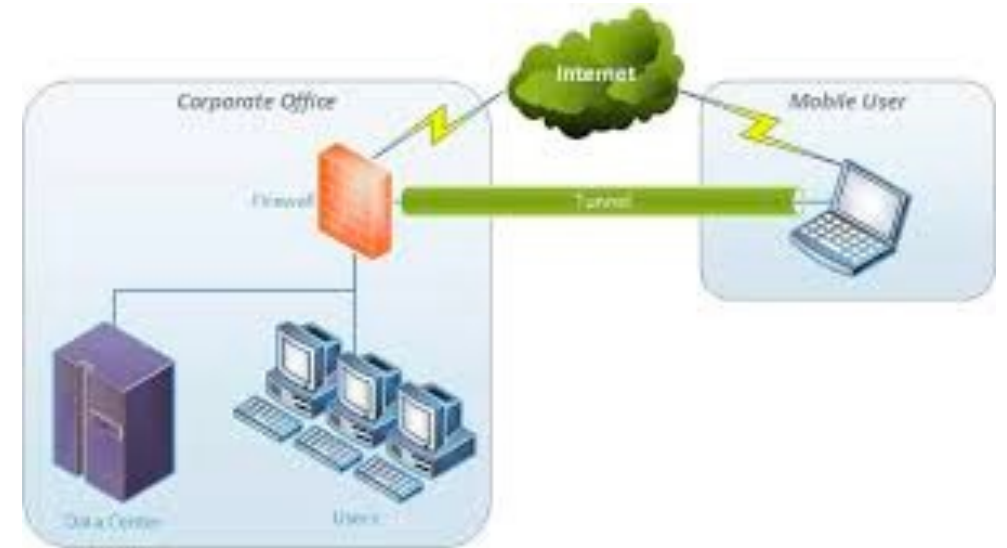




Connecting to the GRID@CBS cluster



COLUMBIA-VPN



- Provides a mean for **secure user access** to Columbia internal services.
- Protects internal CBS systems without requiring complex, error-prone, firewall configurations.
- Creates a ***virtual presence* on Columbia internal networks**, similar to being connected to (secure) campus infrastructure.
- Due to its ability to grant high-value privileges, it requires dual-factor authentication.

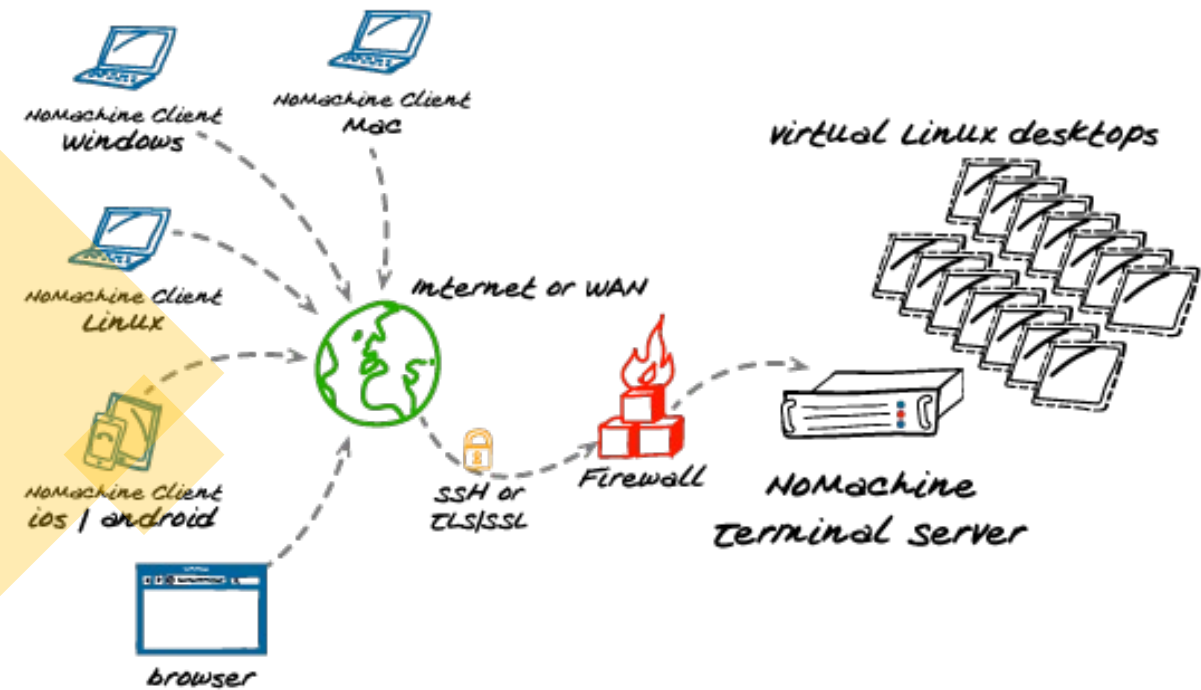
Connecting to COLUMBIA-VPN

- References:
 - CBS-ITG's "CBS Virtual Private Networking"
 - <https://www8.gsb.columbia.edu/itg/vpn>
 - CUIT VPN (scroll down to "How to connect with CUIT VPN using Duo MFA"):
 - https://cuit.columbia.edu/remote-access-services#/cu_accordion_item-11433
- Step 1: Setup and activate Duo MFA
 - a) Install and pair the mobile Duo app, as described in these videos:
<https://cuit.columbia.edu/mfa#/text-1841>
 - b) (optional) Test the MFA setup:
<https://cas.columbia.edu/cas-duo-enroll/duoAuth>
- Step 2: Connect to CUIT VPN gateway
 - a) Go to <https://vpn.cc.columbia.edu> to download and install the Cisco AnyConnect VPN client. At the second password prompt, type "PUSH". Accept the mobile Duo MFA challenge on the phone setup at step 1.
 - b) Open Cisco AnyConnect client and type **vpn.cc.columbia.edu** in the window. Click "Connect"
 - c) Enter your UNI, password, and the word **PUSH** in the field "Duo Action". Click "OK" and accept the mobile Duo MFA challenge.
 - d) ***Alternative* to step 2c above:** if you have problems receiving the Duo authentication challenge on your mobile phone:
 - Open the Duo app and copy the displayed 6 digits numeric code
 - Enter it in the "Duo Action" field **instead of** the word "PUSH"

Connecting to the GRID Interactive Service (“login node”)

- The *GRID Interactive Service* is composed of 3 login servers, but presents a single access point: research.gsb.columbia.edu
- Supports 2 types of interfaces:
 - Virtual desktop
 - Command line terminal
- Requires specialized clients:
 - Nomachine client:
 - Provides both virtual desktop and command line terminal interf.
 - **Supports graphical applications** (Rstudio, MATLAB, etc.)
 - SSH client {Mac Terminal, PuTTY, etc.}
 - Lightweight
 - Provides only command line terminal interface
 - NO SUPPORT for graphical applications
 - (New – VS Code integration)





Nomachine client

- Multi-platform enhanced visual client **optimized for remote access to graphical applications**.
- Supports **virtual desktops** and (rootless) **custom applications** (e.g. terminal emulators).
- **Provides session persistency!** (can disconnect/reconnect to active sessions without losing progress)
- Best for all graphical applications (client-side caching and traffic compression).



Connecting to the GRID via Nomachine client

1. Download and install the Nomachine **Enterprise Client** from:

<https://www.nomachine.com/download-enterprise>

2. Create a new connection template:
 - Give it a friendly name
 - Host: **research.gsb.columbia.edu**
 - Protocol: **NX** (Port: 4000)
 - Authentication: Password



NoMachine Enterprise Client

- ↓ NoMachine Enterprise Client for Windows
- ↓ NoMachine Enterprise Client for Mac
- ↓ NoMachine Enterprise Client for Linux
- ↓ NoMachine Enterprise Client for Raspberry P

[Learn more](#)

Connecting via Nomachine

-- Virtual Desktop --

3. Double click the connection icon just created at step 2 and fill in the authentication information (UNI & password)
4. Double click on “Create a new virtual desktop”
 - (BUG?!) Accept the defaults when prompted... they won't provide the desired config anyway...
5. A virtual desktop running on one of the three servers will appear
 - CTRL+ALT+0 and select the DISPLAY config:
 - Select “Resize remote display”
 - You can now extend your desktop's effective area by resizing the window
 - **To terminate the session**, click your name on the top right corner and select “Log Out...”. Confirm on the popup window.
 - **To disconnect the session** *but leave it active for further reconnection*, close the desktop window from the (X) corner buttons.

Connecting via Nomachine

-- Command Line Terminal --

3. Double click the connection icon just created at step 2 and fill in the authentication information (UNI & password)
 4. Double click on "Create a new custom session"
 1. Select "Run the following command"...
 2. and enter `"/usr/bin/x-terminal-emulator"` in the field
 3. Check "Save this setting in the connection file"
 5. A command line terminal will appear in a floating window
 - **To terminate the session**, type "exit" or CTRL-d, or close the terminal window from the corner buttons.
 - **To disconnect the session** but leave it active for further reconnection, close the **Nomachine** control window from the (x) corner buttons.
- *Mac users need to install XQuartz (www.xquartz.org) and logout/login or restart!!*

Custom session

Application

- ☐ Run the console
- ☐ Run the default X client script on the server
- ☒ Run the following command

Options

- ☒ Run the command in a floating window
- ☐ Run the command in a virtual desktop
- ☒ Save this setting in the connection file

 Logged in as rp2927  Logout

 Back

ENTERPRISE TERMINAL SERVER



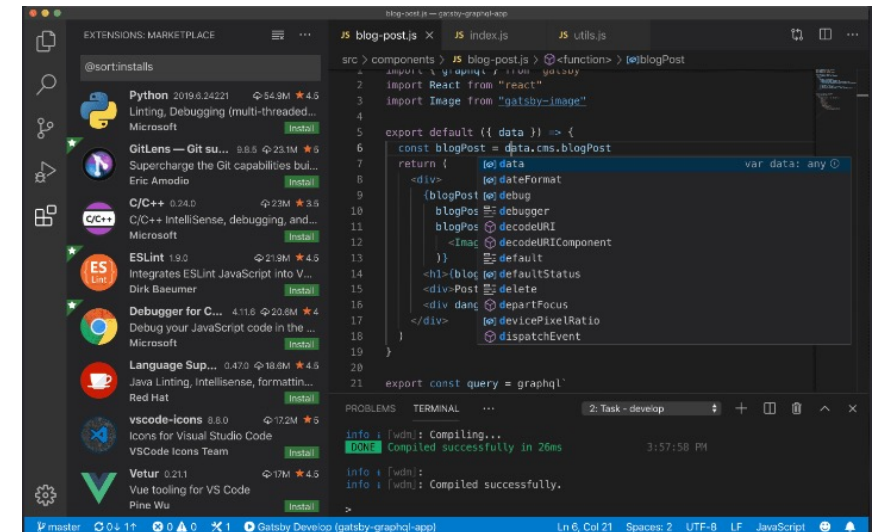
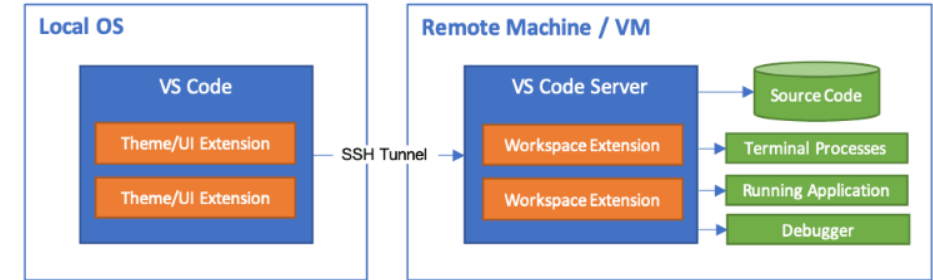


Connecting to the GRID via SSH client

- Supports only command line terminals (no virtual desktops)
 - *Supports graphical applications via X11 tunnel. GUI responsiveness is net inferior to Nomachine client, and thus we recommend the use of the latter solution when GUI support is required.*
- Mac platforms can use the native “Terminal” application:
 - Connect via “ssh UNI@research.gsb.columbia.edu”
- Windows platforms should use PuTTY (www.putty.org)
 - Host: research.gsb.columbia.edu
 - Port: 22 (default)

VS Code Integration

- Download and install VS Code on your laptop.
(<https://code.visualstudio.com/>)
- Install the “Remote -- SSH” extension.
- Go to Settings and change “Remote.SSH: Lockfiles in Tmp” to True
- Create an SSH connection to **research.gsb.columbia.edu** (the Code will install the remote server automatically at the first connect)
 - Install (remotely) the Python extension.
 - Read the “Remote Development using SSH” section of the VS Code docs.
(<https://code.visualstudio.com/docs/remote/ssh>)



Transferring data

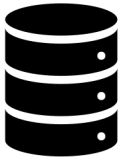
- GRID@CBS uses a **dedicated** data transfer portal to avoid interference with interactive work.
 - Hostname: **researchfiles.gsb.columbia.edu**
 - (Protocol: **SFTP** ; Port: **22**)
- Use your favorite data transfer (sftp) client:
 - Filezilla (<https://filezilla-project.org/>)
 - Cyberduck (<https://cyberduck.io/>)
 - WinSCP (<https://winscp.net/>)
- *For high-volume data transfers use GLOBUS (globus.org)*



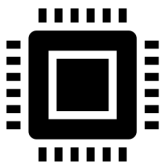
Accessing GRID storage via native file browser

- Can access the GRID storage (home dir, shares, etc.) via the local file browser (Finder or Windows Explorer)
- Mount the Grid home directory via:
 - Mac Finder: Menu “Go” -> “Connect to Server” and enter:
 - `smb://researchfiles.gsb.columbia.edu/`
 - Enter UNI and password when prompted
 - Windows Explorer: “*Map network drive*” using:
 - [\\researchfiles.gsb.columbia.edu\<UNI>](smb://researchfiles.gsb.columbia.edu/<UNI>)
 - Select “Connect using different credentials” and make sure you use your UNI and passwd and not the desktop’s default credentials!
- This method is still under development so expect some instabilities.

Storage at GRID@CBS



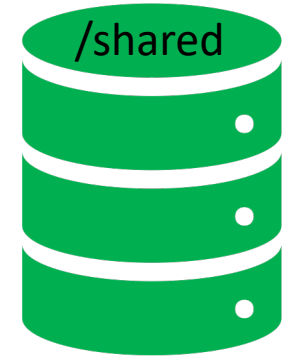
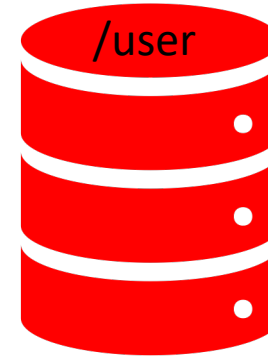
- Disk space vs. memory (RAM) space
 - Disk space:
 - Stores **files**: code, input/output data, logs, config, etc.
 - **Persistent**
 - Controlled by (disk space) **quota**
 - Check quota using the “*lsquota*” command



- Memory (RAM):
 - Array of bytes that the executables use to perform computations.
 - Contains all code variables (structures) *during code execution*.
 - **Volatile**
 - Created when jobs run, *extended during execution as needed (!)* and destroyed at termination.
 - Controlled by the operating system and *limited by the job scheduler*.

Disk Storage at GRID@CBS

- Disk space: three main "areas":
 - Home directory
 - Primary individual disk space – 100GB
 - **/user/UNI**
 - Shared folders (project space)
 - Created for project/group collaborations
 - Shared access by given user group
 - E.g. **/shared/share_B9334**
 - Scratch
 - ***TEMPORARY*** individual workarea – 500GB
 - **/scratch/UNI** (*created by user*)
 - **No protection of any kind!!**
 - **Files not used in 1mo will be automatically deleted**
- Separate quotas apply
- Home & Shared are protected by backup and snapshots (NOT scratch!!)



Filesystem quota

- Default: 100GB for home directories (varies for /shared projects) and 2M files.
- Check with the “lsquota” command:

```
(live)rp2927@researchint01:~$ lsquota
User rp2927 is a member of groups: share_B9334 mysql_rp2927

Disk quotas for user rp2927 :

```

Block Limits								File Limits					Remarks
Filesystem	Fileset	type	blocks	quota	limit	in_doubt	grace	files	quota	limit	in_doubt	grace	
gsb	home	USR	52.62G	100G	110G	0	none	82613	2097152	3145728	0	none	
gsb	work	USR	0	1G	1G	1.084G	none	0	0	0	1	none	
scratch	scratch	USR	882.5G	2T	2.051T	0	none	19289	0	0	0	none	

- Observe both “blocks” (raw space) and “file” (file count) limits!

File formats across platforms

- Various “platforms” (Windows, Mac, Linux) employ different formats for certain files. This will create incompatibilities and generate errors when files are moved between platforms. Certain file formats can be easily converted, some cannot.
- A. Executables (applications and compiled code) are platform specific and *cannot* be converted. One needs to install the appropriate version of the application (Win, Mac, Linux; 32/64bit) or rebuild from source code. Attention to (compiled) modules for modular applications (e.g. Python, R)
- B. Text files (**source code**, configs, **scripts**) might use a different end-of-line (EOL) terminator. Conversion is relatively easy.
 - Frequent mistake: **script files will fail**, errors indicating unexpected “^M” characters at the end of the line.

Beware of Windows EOL characters

- Windows uses a different line terminator that will confuse Linux. The line terminator is an invisible character; thus, you'll not see any difference by simply listing the file!
- **Script files will behave oddly when executed.**
- **Error log files will show “^M” characters at the end of the quoted lines!**
- The “file” command will indicate the file has “CRLF line terminators”

Use the command “dos2unix” to convert text/script files imported from Windows systems.

```
(live)rp2927@researchint01:/shared/share_B9334$ cat test.py
#!/apps/anaconda3/bin/python
print('Testing...')
quit()

(live)rp2927@researchint01:/shared/share_B9334$ ./test.py
-bash: ./test.py: /apps/anaconda3/bin/python^M: bad interpreter: No such file or directory
(live)rp2927@researchint01:/shared/share_B9334$
(live)rp2927@researchint01:/shared/share_B9334$ file test.py
test.py: a /apps/anaconda3/bin/python script, ASCII text executable with CRLF line terminators
(live)rp2927@researchint01:/shared/share_B9334$
(live)rp2927@researchint01:/shared/share_B9334$ cat -v test.py
#!/apps/anaconda3/bin/python^M
print('Testing...')^M
quit()^M
^M
(live)rp2927@researchint01:/shared/share_B9334$ dos2unix test.py
dos2unix: converting file test.py to Unix format...
(live)rp2927@researchint01:/shared/share_B9334$
(live)rp2927@researchint01:/shared/share_B9334$ file test.py
test.py: a /apps/anaconda3/bin/python script, ASCII text executable
(live)rp2927@researchint01:/shared/share_B9334$
(live)rp2927@researchint01:/shared/share_B9334$ cat -v test.py
#!/apps/anaconda3/bin/python
print('Testing...')
quit()

(live)rp2927@researchint01:/shared/share_B9334$ ./test.py
Testing...
(live)rp2927@researchint01:/shared/share_B9334$
```


Data analysis -- A typical workflow

- Requirements:
 - Data transfers
 - File management
 - File editing
 - Running user code or applications (interactively)
 - Run code or applications in batch mode
 - Reading state, logs and code output

Preparing input data

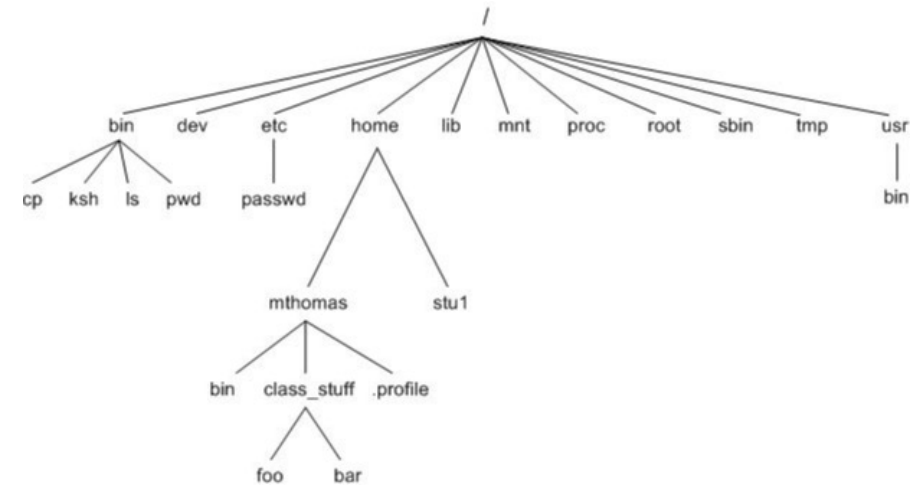


File management on Linux systems



File management on Linux systems

- The Linux file namespace:
 - Hierarchical namespace composed of...
 - Nested folders (directories) and regular files (...and links)
 - Root is (named): `/`
- The Linux filename:
 - A sequence of nested folders, starting at *root* (`/`)...
 - **Separated by (forward) slashes `/`** ...
 - Ending with the file or directory name
 - File and directory names are arbitrary strings of alphanumeric characters.
 - **Case sensitive!**
 - **No concept of type or version built into the system**
 - **Do not use special characters (`~!$%&*()/{ }[ ];`) and avoid blank space.**
- Remote filesystems and namespace mounting:
 - To be accessible, all storage “containers” must be *mounted* into the common namespace tree
 - Uniform access model for local and remote data



```
/user/rp2927/test/HelloWorld.sh  
/shared/share_B9334/data/test01  
/scratch/rp2927/src/Stata15/xstata-mp.tar.gz
```


File management on Linux systems – Absolute and Relative file references

- The “current working directory” is a useful convenience... and an important concept
 - Using (long) full file paths is tedious and error prone
 - E.g. `/shared/share_projectA/credit/market_data/data_by_company/company_monthly_join/2020/07/IBM/sec_filling`
 - Declare a “current directory” and refer to files relative to that location
 - Refer to the **current directory as “.”** and complete the filename reference **relative** to it:
 - E.g. if CWD is: `/shared/share_projectA/credit/market_data/data_by_company/company_monthly_join/2020/07/IBM`
 - Use `./sec_filling` or `./policy/environmental` to access the respective sub-directories and files therein
 - Commands:
 - **cd** (“change (working) directory”)
 - **pwd** (“print working directory”)
 - Special notations:
 - Current directory: `“.”`
 - Parent directory: `“..”`
 - Home directory: `“~”`

Browsing the namespace

...without a visual browser

- Moving (the current directory) around:
 - Down: `cd subfolder` OR `cd subdir1/subdir2/subdir3`
 - Up: `cd ..` OR `cd ../../..`
- Listing directory content:
 - `ls subdir`
 - `ls subdir1/subdir2`
 - `ls` (OR `ls .`) Lists the current directory
 - `ls ..` Lists the parent directory
 - `(ls -larth subdir)` (ls options: long, all, by time, reverse order, human units)
- Use search patterns to limit the scope of listings to relevant files:
 - `ls ./IBM/sec_filling/*13H*.dat`

Files and directories. Attributes and mode.

- File/dir attributes
 - List using the “long” format (`ls -l`)
 - List “hidden” files (filenames starting with “.”) (`ls -a`)
- Mode (file protection)
 - 12bit mask – format: “`f/d/l UUU GGG OOO`”
 - f/d/l = {-|d|l} file type (file/directory/link)
 - U/G/O = user/group/other
 - rwx = Read Write Execute
- Command: `chmod` (“change mode”)
 - `chmod mode FileDirName`
 - `mode` = [ugo]{+|=}{rwx}
 - Ex.: `chmod +x exec.sh` (memorize it !!)

Mode	(links)	Owner	Group	Size	Timestamp	Name
[(live)rp2927@researchint01:~\$ ls -la						
total 6345						
drwx-----	79	rp2927	rp2927	16384	Jan 12 11:45	.
drwxrwxr-x	875	root	root	32768	Jan 9 04:49	..
drwxrwxr-x	4	rp2927	rp2927	4096	Jul 3 2019	accounts
drwxrwxr-x	3	rp2927	rp2927	4096	Jul 6 2018	.anaconda
drwxrwxr-x	3	rp2927	rp2927	4096	Jul 6 2018	.astropy
drwxrwxr-x	2	rp2927	rp2927	4096	Jan 7 21:23	B9334
-rw-----	1	rp2927	rp2927	7739	Jan 12 11:33	.bash_history
-rw-----	1	rp2927	rp2927	18	Jul 3 2018	.bash_logout
-rw-----	1	rp2927	rp2927	260	Oct 8 15:57	.bash_profile
-rw-----	1	rp2927	rp2927	797	Nov 17 21:21	.bashrc
-rw-r--r--	1	rp2927	rp2927	1329	Dec 14 2018	_bashrc_anaconda3
drwxrwxr-x	3	rp2927	rp2927	4096	Oct 26 18:22	bin
drwxrwxr-x	23	rp2927	rp2927	4096	Oct 7 18:28	.cache
drwxrwxr-x	2	rp2927	rp2927	4096	Mar 25 2019	compare_env
drwxrwxr-x	4	rp2927	rp2927	4096	Sep 6 2018	.conda
-rw-rw-r--	1	rp2927	rp2927	66	Nov 17 21:22	.condarc
drwxr-xr-x	30	rp2927	rp2927	4096	Dec 2 12:32	.config
-rw-rw-r--	1	rp2927	rp2927	51	Oct 15 2019	cplex.log
-rwxrwxr-x	1	rp2927	rp2927	8464	Feb 7 2020	crash
-rw-rw-r--	1	rp2927	rp2927	31	Feb 7 2020	crash.c
drwx-----	3	rp2927	rp2927	4096	Sep 12 2018	.dbus
drwxr-xr-x	2	rp2927	rp2927	4096	Nov 13 20:20	Desktop

Creating/destroying content

- Make a new directory (**mkdir**)
 - **mkdir** *dirName*
- Delete (remove) directory (**rmdir**)
 - **rmdir** *dirName* (will work only if dir is empty!)
- Create a new file
 - **touch** *fileName*
 - Or use a file editor
- Remove a file (**rm**)
 - **rm** *fileName*
- Copy a file (**cp**)
 - **cp** *oldFileName newFileName*
- Move (rename) a file (**mv**)
 - **mv** *oldFileName newFileName*

Displaying file content

...without a file browser (!)

- Displaying the content of a file uses a strangely named command: **cat** (“concatenate and print”)
 - **cat** *fileName*
- Paging through screenfuls of file content has an even more bizarre name choice: **less** (OR **more**) (Geek humor: “less is more”)
 - **less** *fileName*
 - (**more** *fileName* ; “more” is the older version of this Unix utility)
 - “less” Subcommands:
 - <space> page forward ; page backward
 - keyboard arrows move one line at a time
 - <q> exit
 - </string> find ; <n> next match forward ; <N> next match backward
 - <G> go to end ; <g> go to beginning

File editors

- Vi
- Nano
- Emacs
- [your favorite here]

Overview of nano's shortcuts

The editor's keystrokes and their functions

File handling

Ctrl+S Save current file
Ctrl+O Offer to write file ("Save as")
Ctrl+R Insert a file into current one
Ctrl+X Close buffer, exit from nano

Editing

Ctrl+K Cut current line into cutbuffer
Alt+6 Copy current line into cutbuffer
Ctrl+U Paste contents of cutbuffer
Alt+T Cut until end of buffer
Ctrl+] Complete current word
Alt+3 Comment/uncomment line/region
Alt+U Undo last action
Alt+E Redo last undone action

Search and replace

Ctrl+Q Start backward search
Ctrl+W Start forward search
Alt+Q Find next occurrence backward
Alt+W Find next occurrence forward
Alt+R Start a replacing session

Deletion

Ctrl+H Delete character before cursor
Ctrl+D Delete character under cursor
Alt+Bsp Delete word to the left
Ctrl+Del Delete word to the right
Alt+Del Delete current line

Operations

Ctrl+T Execute some command
Ctrl+J Justify paragraph or region
Alt+J Justify entire buffer
Alt+B Run a syntax check
Alt+F Run a formatter/fixer/arranger
Alt+: Start/stop recording of macro
Alt+; Replay macro

Moving around

Ctrl+B One character backward
Ctrl+F One character forward
Ctrl+← One word backward
Ctrl+→ One word forward
Ctrl+A To start of line
Ctrl+E To end of line
Ctrl+P One line up
Ctrl+N One line down
Ctrl+↑ To previous block
Ctrl+↓ To next block
Ctrl+Y One page up
Ctrl+V One page down
Alt+\ To top of buffer
Alt+/ To end of buffer

Special movement

Alt+G Go to specified line
Alt+] Go to complementary bracket
Alt+↑ Scroll viewport up
Alt+↓ Scroll viewport down
Alt+< Switch to preceding buffer
Alt+> Switch to succeeding buffer

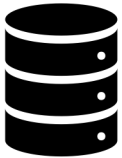
Information

Ctrl+C Report cursor position
Alt+D Report word/line/char count
Ctrl+G Display help text

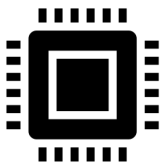
Various

Alt+A Turn the mark on/off
Tab Indent marked region
Shift+Tab Unindent marked region
Alt+N Turn line numbers on/off
Alt+P Turn visible whitespace on/off
Alt+V Enter next keystroke verbatim
Ctrl+L Refresh the screen
Ctrl+Z Suspend nano

Storage at GRID@CBS



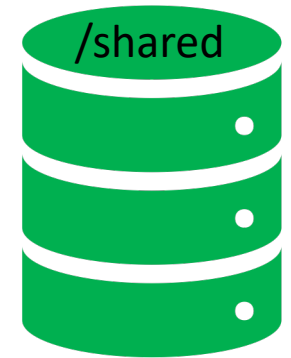
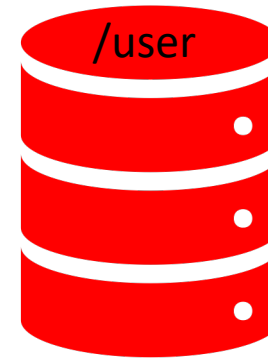
- Disk space vs. memory (RAM) space
 - Disk space:
 - Stores **files**: code, input/output data, logs, config, etc.
 - **Persistent**
 - Controlled by (disk space) **quota**
 - Check quota using the “*lsquota*” command



- Memory (RAM):
 - Array of bytes that the executables use to perform computations.
 - Contains all code variables (structures) *during code execution*.
 - **Volatile**
 - Created when jobs run, *extended during execution as needed (!)* and destroyed at termination.
 - Controlled by the operating system and *limited by the job scheduler*.

Disk Storage at GRID@CBS

- Disk space: three main "areas":
 - Home directory
 - Primary individual disk space – 100GB
 - **/user/UNI**
 - Shared folders (project space)
 - Created for project/group collaborations
 - Shared access by given user group
 - E.g. **/shared/share_B9334**
 - Scratch
 - ***TEMPORARY*** individual workarea – 500GB
 - **/scratch/UNI** (*created by user*)
 - **No protection of any kind!!**
 - **Files not used in 1mo will be automatically deleted**
- Separate quotas apply
- Home & Shared are protected by backup and snapshots (NOT scratch!!)



Filesystem quota

- Default: 100GB for home directories (varies for /shared projects) and 2M files.
- Check with the “lsquota” command:

```
(live)rp2927@researchint01:~$ lsquota
User rp2927 is a member of groups: share_B9334 mysql_rp2927

Disk quotas for user rp2927 :

```

Block Limits								File Limits					Remarks
Filesystem	Fileset	type	blocks	quota	limit	in_doubt	grace	files	quota	limit	in_doubt	grace	
gsb	home	USR	52.62G	100G	110G	0	none	82613	2097152	3145728	0	none	
gsb	work	USR	0	1G	1G	1.084G	none	0	0	0	1	none	
scratch	scratch	USR	882.5G	2T	2.051T	0	none	19289	0	0	0	none	

- Observe both “blocks” (raw space) and “file” (file count) limits!



Brief introduction to Linux shell (bash)

Linux shell

- Main command interpreter that drives the user-system interaction in command line interfaces (terminals).
- Processes commands either entered by the user or read from a file (script).
 - Controls user dialogue in command line terminals.
 - **Runs scripts and starts applications or user code.**
- Provides:
 - Simple data structures (variables, arrays)
 - Conditionals (if, case)
 - Loops (while, until, for)
 - Simple integer arithmetic
 - I/O (read, write, pipes, redirects)
 - Functions
 - Job control
 - Aliases
- Runs non-built-in commands via fork and execute

Data structures

Operations with strings

- Variable assignment and access:
 - VAR="string" (*no space around the operator!*)
 - echo \${VAR}
- Length of string: \${#VAR}
- Concatenation: VAR9="\${VAR1} \${VAR2}"
- Substring: \${VAR:position:length}
- String deletions:
 - Delete shortest substring match from front: \${VAR#substring}
 - Delete shortest substring match from back: \${VAR%substring}
 - For *longest* match double the operator: ##, %%
- String substitutions: \${VAR/pattern/replacement} (double the operator for *all* matches)
- (simple) Matching patterns:
 - * - any string
 - ? – one character
 - [...] – any character from the enumeration. Use a dash to declare a range (e.g. 0-9), or !/^ to negate

Conditional statements

```
if TEST ; then
    COMMANDS ;
elif MORE_TESTS ; then
    MORE_COMMANDS ;
else
    OTHER_COMMANDS ;
fi
```

- Examples of TESTS:
 - [-e FILE] -- True if file exists
 - [-z STRING] – True if STRING is zero
 - [-n STRING] – True if string is non-zero
 - [STR1 *OP* STR2] – where *OP* can be ==, !=, <, >, =~
 - [ARG1 *OP* ARG2] – where *OP* can be -eq, -ne, -lt, -le, -gt, -ge
- Combining expressions:
 - [! *EXPR*] – negation (logical NOT)
 - [*EXPR1* -a *EXPR2*] – logical AND
 - [*EXPR1* -o *EXPR2*] – logical OR

Arithmetic evaluation and command substitution

- `(( EXPRESSION ))`
- `VAR=$(( EXPRESSION ))`
- Integer arithmetic!
- Operators:
 - `++, --, !, *, /, %, +, -, <, >, <=, >=, ==, !=, ||, &&, =, +=, -=, *=, /=`, and more.
- `$(command)`
- ``command``
- Bash executes the command and returns its output (can be used for assignment). E.g.:
 - `TXT_FILES=$(ls *.txt)`

Loops

```
for NAME in LIST ; do  
    COMMANDS  
done
```

```
while TEST ; do  
    COMMANDS  
done
```

```
until TEST ; do  
    COMMANDS  
done
```

- Examples:
 - for FILE in `ls \*.gz`; do gunzip \$FILE ; done
 - a=1; while ((a <= 10)); do echo \$a ; ((a++)); done
 - b=10; until ((x == 0)); do echo \$b ; ((b/=2)); done
 - for ((i=1 ; i<=10 ; i++)); do echo \$i; done
- Breaking out of loops:
 - `continue` – jumps to the next iteration
 - `break` – exits the loop

Functions and positional parameters

- Function declaration:
 - `function FUNC { Commands; }`
 - `FUNC () { Commands; }`
- Function calling:
 - `FUNC param1 param2 ...`
 - (positional) parameters can be accessed via `$1`, `$2`, ...
- *(!) Make sure the function is defined before calling it.*
- Positional parameters access:
 - `$N` – the N-th positional parameter from the script/function call
 - `$@` -- all the positional parameters (as a list)
 - `$#` -- the number of positional parameters

A few more useful built-in commands and utilities

- “**echo** *args*” – prints arguments to stdout (terminal)
 - “echo -n” suppresses the trailing newline
 - “echo -e” interprets backslash-escaped characters (\t, \n, etc.)
- “**read** *vars*” – reads user input into list of variables
 - E.g.: read -p “Please enter full name and UID: “ first last uid
- “**grep** *pattern file*” – returns all lines from *file* matching the *pattern*
- “**cut**” – selects portions of (file) lines
- “**awk**” – a very powerful stream editor
 - simple & effective column selections
 - floating point math
- “**sed**” – another versatile stream editor
 - easy to make simple file edits

Streams and Redirection

- Every process has three streams available by default: stdin, stdout and stderr.
 - Unless specified otherwise the shell (scripts) reads from stdin and writes to stdout.
 - An interactive shell defaults to all three streams connected to the calling terminal.
 - The user has the ability to change that...
- Input redirection: “*command < file*”
 - Special case - “here document”:
command <<terminator
...
terminator
- Output redirection: “*command > file*” (or “*command >> file*” to append)
 - Special case: *2>&1* redirects stderr to stdout
- Pipes: “*command1 | command2*”

Aliases

- Shortcuts used to facilitate user interaction or to specify default options to a command. (Must appear as the 1st work of a command)
- Define/undefine using the “alias”/”unalias” built-in command, e.g.:

```
>> alias l='ls -l -color=auto'
```
- List all using “alias” without arguments

Accessing local system documentation

- Probably the most important command is...
 - Print the documentation of a command, tool or utility – the “manual” (**man**)
 - **man** *command*
- Examples:
 - **man** *ls*
 - **man** *rm*
 - **man** *man*
 - **man** *bash* --- *This command will display the manual of the bash shell (a must read) ---*

Scripts

- Plain text file containing shell commands (built-in or external).
- Must be executable (!)
- Comment lines start with #
- “Shebang”: special character pair “#!” placed on the very first line of the script, followed by the file name of an “interpreter” to be invoked by the shell. (Very important!) Examples:
 - #!/bin/bash
 - #!/apps/anaconda3/bin/python
 - #!/user/rp2927/.conda/envs/tf_gpu/bin/python
 - #!/apps/bin/Rscript
- Called by file name and accepting positional parameters
- Have access to all *environmental variables* of the calling shell (!!)

Configuration (startup) files

- Immediately after login the shell reads in a few configuration files to setup your environment, before prompting you for commands.

- `~/.profile` for interactive shells
- `~/.bashrc` for non-interactive shells

```
((live)rp2927@researchint01:~$ cat .bashrc
# .bashrc

# Source global definitions
if [ -f /etc/bashrc ]; then
    . /etc/bashrc
fi

# Uncomment the following line if you don't like systemctl's auto-paging feature:
# export SYSTEMD_PAGER=

# User specific aliases and functions
alias l='ls -lh'
alias la='ls -lah'
alias lt='ls -larth'
alias sgr='sge_run'
alias sgrt='sge_run --grid_q=test.q'

# >>> conda initialize >>>
# !! Contents within this block are managed by 'conda init' !!
__conda_setup="$(('/apps/anaconda3/bin/conda' 'shell.bash' 'hook' 2> /dev/null)"
if [ $? -eq 0 ]; then
    eval "$__conda_setup"
else
    if [ -f "/apps/anaconda3/etc/profile.d/conda.sh" ]; then
        . "/apps/anaconda3/etc/profile.d/conda.sh"
    else
        export PATH="/apps/anaconda3/bin:$PATH"
    fi
fi
unset __conda_setup
# <<< conda initialize <<<

((live)rp2927@researchint01:~$
```

```
((live)rp2927@researchint01:~$ cat .bash_profile
# .bash_profile

# Get the aliases and functions
if [ -f ~/.bashrc ]; then
    . ~/.bashrc
fi

# User specific environment and startup programs

PATH=$PATH:$HOME/.local/bin:$HOME/bin

export PATH

# Add the SGE man pages to the path
export MANPATH=/opt/nlge/man
((live)rp2927@researchint01:~$
```

Script examples

See: `/shared/share_B9334/GRID/HelloWorld.sh`

```
[(live)rp2927@researchint01:~$ cat /shared/share_B9334/GRID/HelloWorld.sh
#!/bin/bash

# Test script illustrating a few basic elements of bash scripting...

# Command substitution...
HOST=$(hostname)

# add an environment variable ($NSLOTS) to the mix and concatenate the elements...
echo Running on node $HOST on $(date) using $NSLOTS cores

# certain important elements are available via commands or as environment variables...
echo "Working directory: $(pwd) ...or: $PWD"

# For batch jobs the scheduler makes available certain control variables...
echo -e "Here are the task control vars:\n$(env | grep SGE_TASK)"

# Access a positional argument...
echo The Weather is $1

# ... out of how many?
echo "...and the full list of command parameters:"

while [ -n "$1" ] ; do
    echo "param: $1"
    shift
done
```



```
mirror_mod = modifier_ob.  
#set mirror object to mirror  
mirror_mod.mirror_object =  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True  
  
#selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
  
print("please select exactly")  
  
-- OPERATOR CLASSES --
```

```
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"
```

```
context):  
context.active_object is not None
```

Running applications

Running applications (interactively) ... the short version

- Simply type the application name or rather its designated (start) command invocation; e.g.:
 - matlab
 - R
 - stata
- For graphical interfaces we constructed separate invocations by adding an “x” before the regular command name: “xmatlab”, “xstata”
 - Must use the NoMachine client

Running applications

... on an HPC cluster

- The "what" and the "where":
 - Resource allocations
 - Memory (RAM) and CPU (processing) cores
 - Remote worker nodes
 - Requests and limits
 - Machine boundaries
 - Allocations, excess and quotas
 - Contention and scheduling
 - Interactive vs batch workloads
- Containing the complexity -- the GRID wrapper
- "GRID" options

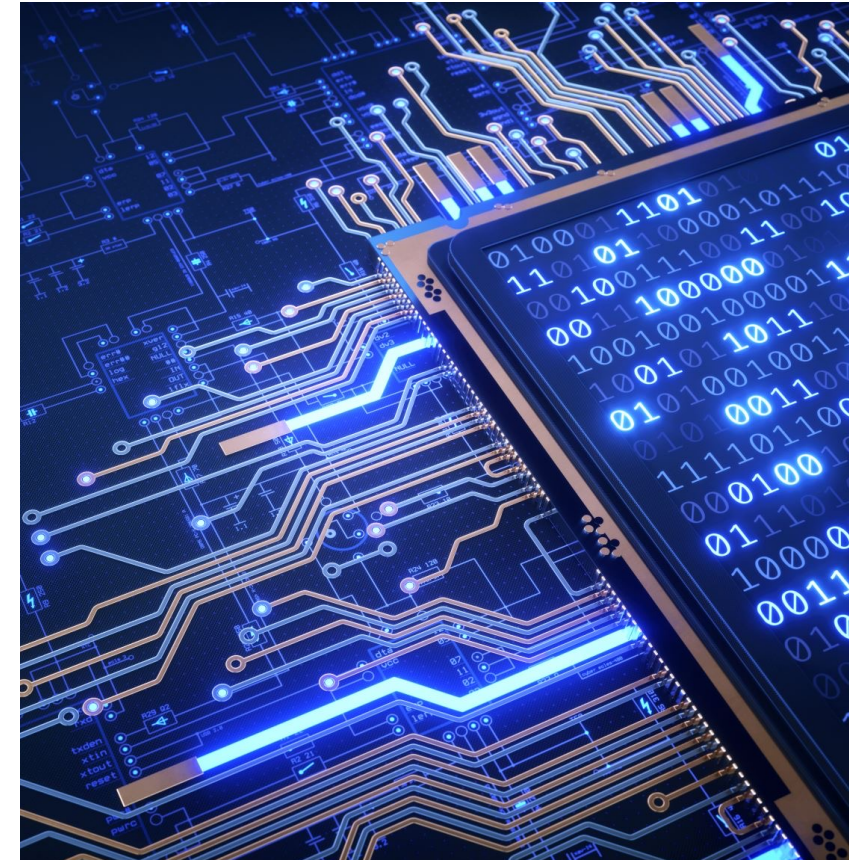
How does the system respond to our commands?

- We type a string of words on the command line:
 - *keyword options parameters*
 - Ex. `stata -b do filename &`
- What is the system doing with that information?
 - It takes the first word ("*keyword*"), *searches for a file with that name* and executes it!
 - Passes the whole command line content to that executable.
- Who controls all that?
 - The "shell" – the program that runs the conversation that occurs at the command prompt immediately after login.
 - By default, we run the "bash" shell.
- Where is the shell searching a match for the first command word?
 - Unless told explicitly (using a full filepath!) the shell uses an *environment variable* named **PATH**, *containing an ordered list of directories*.
 - Ex.: `echo $PATH`
`/apps/anaconda3/condabin:/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/apps/wrappers:/apps/bin:/opt/n1ge/bin/lx24-
amd64:/user/rp2927/.local/bin:/user/rp2927/bin`

Running applications

... on the GRID cluster

- Given the shell's search of a match for an arbitrary string in an ordered list of locations, we can:
 - Add new commands – e.g. the “x” versions of GUI applications vs their command-line counterparts (e.g. `cmdline_matlab == “matlab –nodesktop –nojvm”`)
 - **Overload** existing commands(!) – in fact, all the application invocations you’ve seen so far are “altered”.
- **The “wrapper” script** -- the special script that “intercepts” all GRID application invocations:
 1. *Expands the interface adding GRID specific options (resource allocations, operating modes, etc.)*
 2. *Runs the payload remotely on a selected worker node*



GRID@CBS Command Syntax

Start with the *publicly available standard application invocation* and insert all GRID options after the application/commandName and before any application options or parameters.



The diagram shows a sequence of three blue boxes connected by downward-pointing arrows. The first box contains the instruction to insert GRID options. The second box shows the command syntax 'appName applicOptions applicParameters' with 'gridOptions' written in yellow above 'applicOptions' and a bracket indicating its insertion point. The third box shows the final syntax with 'gridOptions' inserted between 'appName' and 'applicOptions'.

```
appName applicOptions applicParameters
```

```
appName gridOptions applicOptions applicParameters
```

Examples

- matlab --grid_mem=20G -batch myscript
- stata --grid_mem=15G
- stata --grid_mem=25G -b do myfile.do
- xstata --grid_ncpus=8 --grid_mem=40G
- R --grid_ncpus=4 --grid_mem=50G
- Rscript --grid_mem=50G myscript.R

```
mirror_mod = modifier_ob.  
#set mirror object to mirror  
mirror_mod.mirror_object =  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True  
  
#selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
  
print("please select exactly  
  
-- OPERATOR CLASSES --
```

```
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"
```

```
context):  
context.active_object is not None
```

Running user
code

Running user code...

- User code can be:
 - a. Compiled code (e.g. C/C++)
 - The code is compiled (and linked) into an *executable* file
 - b. Interpreted code (Python, Julia, bash, MATLAB, STATA, R, etc.)
 - Running interpreted code requires an *interpreter* (application) that must be named explicitly
 - c. *Control script (e.g. bash)
 - A special case where the interpreter is started implicitly
- Running the code requires:
 - a. Nothing special (the “code” itself is executable)
 - b. Naming (calling/starting) an appropriate interpreter (application)
 - c. Nothing special, as the shell (interpreter) is what starts the processing chain
- However, on HPC installations we also need:
 - A mean to define resource requests
 - A mean to dispatch workloads to remote worker nodes

Don't we already know how to?

- We have seen already a few methods:
 - `matlab --grid_mem=20G -batch myscript`
 - `stata --grid_mem=25G -b do myfile`
 - `Rscript --grid_mem=50G myscript`
 - ... they all work because we modified the applications!
- How about Python?
 - Maybe “`python myScript.py`”?...
 - No!.... the default system configuration interferes with us
 - How about a made-up command then?...
 - Yes!... “`anapy3 --grid_mem=25G myScript.py`”
- How about my C++ code or my bash, Ruby, Julia, etc. script?...
 - How to request resources? How to make sure the payload is sent to a worker node?
- We need a comprehensive approach!

Running an arbitrary user script

- All starts with the shell!
 - The shell (bash) provides a powerful and versatile programming environment, provisioned with logic test, loops, I/O control, integer arithmetic and more.
 - Consider using it to control your analysis workflow
- A shell special feature: the “shebang”
 - Magic character sequence starting with “#!” *on the first line of the shell script*
 - The shell interprets the rest of the line as describing the absolute location of an executable to start
 - The shell starts this executable (interpreter), ignores processing the rest of the file, and instead passes it to this executable as input
- Make sure the script is **executable!!** (remember chmod?)
 - `chmod +x scriptName`



Shebang examples

- `#!/bin/bash`
 - Runs the bash shell. A bit redundant unless you run on a system with a different default shell)
- `#!/apps/anaconda3/bin/python`
 - Runs the system installed Anaconda Python3 interpreter
- `#!/user/rp2927/.conda/envs/tf_gpu/bin/python`
 - Runs the Python interpreter from my private environment named tf_gpu
- `#!/apps/bin/Rscript`
 - Starts the Rscript interpreter for running R code

Running arbitrary code/script

- We can now run arbitrary interpreted (Python, Julia, ruby, etc.) or compiled (C/C++/Fortran) code, but ...
- If we would invoke the compiled code or script name, as we did with applications, the code or script will execute locally on the interactive nodes, which is highly undesirable, counter-productive and even disruptive.
 - (Remember, the common applications were wrapped and transparently modified to run remotely)
- Moreover, the GRID options won't be recognized!
- To use all the facilities and resources of the cluster, we need a special tool

Executing arbitrary code, scripts or applications on GRID@CBS

- To use all the facilities and resources of the cluster, use:

grid_run

- grid_run is a general-purpose handler useful for **both scripts and user-built executables** (compiled code), as well as for arbitrary applications.
- grid_run accepts all GRID options
- grid_run uses the cluster scheduler to dispatch the workload to a lightly used node
- (grid_run is in fact the same wrapper code that processes installed applications)



Syntax:

grid_run *gridOptions* **userScriptCodeApp** *userOptions* *userArguments*

*The location of **userScriptCodeApp** must be explicit!

Examples – Bash script

Bash script: test.sh

```
#!/bin/bash
HOST=$(hostname)
echo Running on $HOST at $(date)
echo Weather is $1
```

`grid_run --grid_mem=10G ./test.sh fair`

```
Running on research01 at Tue 12 Jan 2021 06:57:47 PM EST
Weather is fair
```

*grid_run will attempt to locate the [userScript](#) (here: test.sh) using the same rules as the running shell: either via an explicit location or using the PATH. As user scripts are rarely present in the directories included in the PATH (unless the user customized it), it is required that the grid_run invocation contain an **explicit location** for the userScript!! (here: ./ = the local directory). Otherwise, a “file not found” exception will occur.*

Examples – Python script

Python script: test.py

```
#!/apps/anaconda3/bin/python  
print('Testing...')
```

`grid_run` `--grid_mem=5G` `./test.py`

Testing...

Running applications or user code -- summary

Two formats of calls:

1. **application** *grid_options* *applicationOptions* *applicationArguments*
2. **grid_run** *grid_options* **application/code** *appcodeOptions* *appcodeArguments*

Application or user code	Run with application handler	Run with grid_run	Comments
MATLAB	<u>matlab <grid_options> -batch myScript</u>	grid_run --grid_mem=20G matlab --batch myScript	-- Do not include type and location for ./myScript.m !! -- The “matlab” special handler takes care of minimum mem allocation automatically, but grid_run needs an explicit request.
STATA	<u>stata <grid_options> -b do ./myScript.do</u>	grid_run <grid_options> stata -b do ./myScript.do	-- Type and location for myScript.do are optional
R (Rscript)	<u>Rscript <grid_options> ./myScript.R</u>	grid_run <grid_options> Rscript ./myScript.R	-- Location is optional, but type is not , for myScript.R
Python	anapy3 <grid_options> ./myScript.py	<u>grid_run <grid_options> ./myScript.py</u>	-- Both script type and location are required for grid_run, but anapy3 can do without location -- grid_run requires the interpreter’s location in the script’s shebang
bash, Julia, R, Mathematica, etc.	N/A	<u>grid_run <grid_options> ./myScript.sh</u>	-- Both script type and location are required -- Interpreter location in shebang

Preferred formats are underlined

GRID@CBS Options

- All GRID options start with “**--grid_**”
- Resource requests:
 - **--grid_ncpus=n** Number of CPU cores (default n=1)
 - **--grid_mem=nG** Amount of memory (default: 4GB)
 - **--grid_long** Unlimited execution time
 - **--grid_gpu** GPU accelerated system
- Execution mode:
 - **--grid_submit=batch|interactive** (default: interactive)
- Notification:
 - **--grid_email=“myemail@columbia.edu”** Will send an email notification at job completion (batch).
- Array jobs:
 - **--grid_array=<start>-<end>[:<step>][/maxConc]**
 - **--grid_hold=<jobId>**

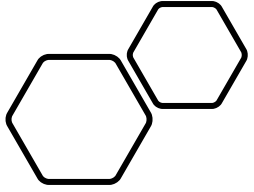


Running production
workloads (batch)

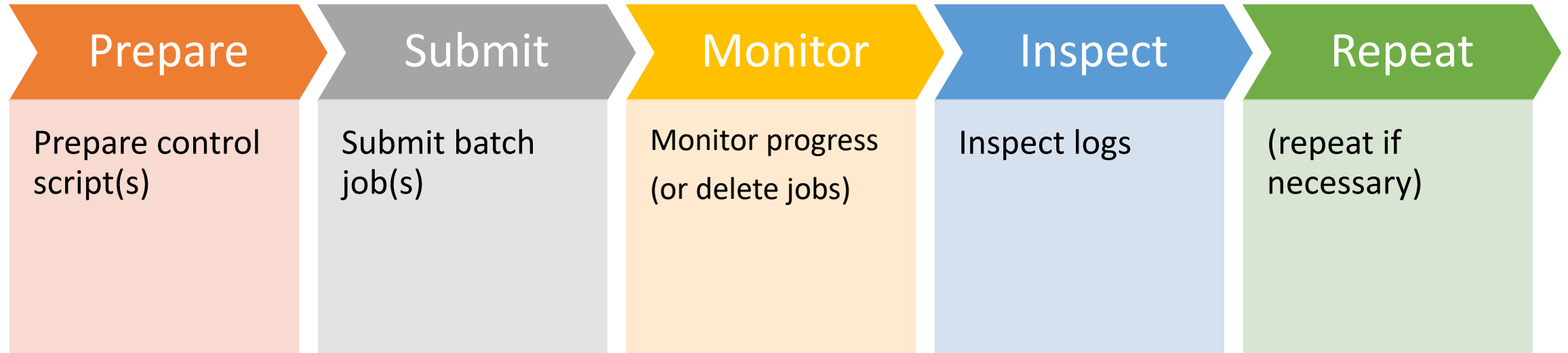
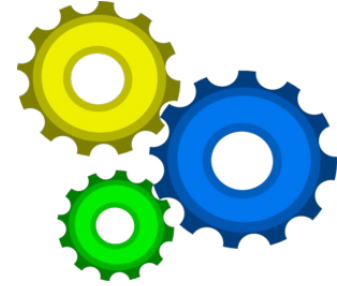
Batch computing

- A classic workflow originating in the mainframe era
 - **Submit** a bundle of payloads (**job requests**) and let the system organize (**schedule**) its execution
 - **Unattended** execution
 - **Fully automated**
- Must prepare a script that fully controls each job
- Must include all input parameters, input and output filenames
- All messages produced by the job execution (outputs and errors) will be captured in log files
- The scheduler (SGE) will organize the execution of all submitted jobs, optimizing GRID resource utilization and global response time.





Batch Workflow



Batch
Workflow:
#1 Prepare

A thick, hand-drawn orange vertical line is positioned to the right of the text, extending from the top of the text area to the bottom of the slide.

Batch Workflow: #2 Submit

Use the same syntax but include the `--grid_submit=batch` GRID option

- Submit the job to the scheduler by using the GRID option: `--grid_submit=batch`
The syntax used so far for running applications or user code remains the same!

Submit an application

- ***applicName*** `--grid_submit=batch` *otherGridOptions* *applicOptions* *applicArgs*

Submit own code (Python or C/C++)

- ***grid_run*** `--grid_submit=batch` *otherGridOptions* *userScript* *userScriptOptions* *userScriptArgs*

```

(live)rp2927@researchint01:/shared/share_B9334/Razvan$
(live)rp2927@researchint01:/shared/share_B9334/Razvan$ grid_run --grid_submit=batch ./HelloWorld.sh
Your job 6585326 ("HelloWorld.sh") has been submitted
(live)rp2927@researchint01:/shared/share_B9334/Razvan$ qstat
job-ID   prior    name         user          state submit/start at     queue                          slots ja-task-ID
-----
6585326  0.00000 HelloWorld rp2927        qw    01/26/2022 13:50:04                          1
(live)rp2927@researchint01:/shared/share_B9334/Razvan$
(live)rp2927@researchint01:/shared/share_B9334/Razvan$ qstat
job-ID   prior    name         user          state submit/start at     queue                          slots ja-task-ID
-----
6585326  0.56000 HelloWorld rp2927        r     01/26/2022 13:50:07  debian.q@research117.grid.gsb  1
(live)rp2927@researchint01:/shared/share_B9334/Razvan$
(live)rp2927@researchint01:/shared/share_B9334/Razvan$
(live)rp2927@researchint01:/shared/share_B9334/Razvan$ qstat
(live)rp2927@researchint01:/shared/share_B9334/Razvan$
(live)rp2927@researchint01:/shared/share_B9334/Razvan$
(live)rp2927@researchint01:/shared/share_B9334/Razvan$

```

Batch Workflow: #3 Monitor

- The scheduler provides specific tools for monitoring and controlling its activity:
 - **Get job status (qstat)**
 - **Delete a job (qdel)**
 - ***Get cluster info (qhost)**
 - (suspend, resume, modify, resubmit, etc. – not used here)
- **When submitted each job receives a **jobID****

Batch Workflow Monitoring

- Requesting status (qstat)
 - qstat (list the status of all the jobs *owned by the caller*)
 - qstat -j <jobID> (list the **detailed** status of the job with the provided jobID)
 - qstat -u "*" (list **all** jobs)
- States:
 - r – running
 - qw – queued waiting
 - t – transferring
 - d – deleting
- Find reason for job waiting
 - qstat -j <jobID> *If job is in "qw" state, the reason for waiting will be displayed at the end of a long list of job parameters. Unfortunately, it is presented in quite a non-obvious way. (Send it to support for interpretation)*



Batch Workflow: #3' Deleting

- Delete a job (**qdel**)
 - `qdel <jobID>`
 - `qdel -u $(whoami)` Delete **all** jobs owned by the caller.
Use with much care!!



Batch Workflow: #4 Inspect logs

- The logs are created in the **same directory from where the job was submitted!**
- The scheduler will collect all the messages generated by the job in 4 log files (not including data/output files created by your code):
 - `<jobName>.o<jobID>` *stdout (standard output) Regular program output*
 - `<jobName>.e<jobID>` *stderr (standard error) Error messages*
 - `<jobName>.po<jobID>` *Empty*
 - `<jobName>.pe<jobID>` *Empty*
- Find *jobName* and *jobID* in the **qstat** output
- The logs are essential for understanding what happened to the job. **Always check them!** Support will always ask for them, as well.

Batch Workflow: Log files examples

```
[rp2927@researchgrid B9334]$ qstat
[rp2927@researchgrid B9334]$
[rp2927@researchgrid B9334]$ ls -lrt
total 32
-rwxrwxr-x 1 rp2927 578103723 207 Feb  7 10:26 HelloWorld.sh
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:26 bash-bat.po4949853
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:26 bash-bat.pe4949853
-rw-r--r-- 1 rp2927 578103723 234 Feb  7 10:26 bash-bat.o4949853
-rw-r--r-- 1 rp2927 578103723  61 Feb  7 10:26 bash-bat.e4949853
[rp2927@researchgrid B9334]$
[rp2927@researchgrid B9334]$ cat bash-bat.o4949853
Running on node: research48
Working directory: /user/user1/rp2927/B9334
Is there anybody out there?...
Here are the task control vars:
SGE_TASK_STEPSIZE=undefined
SGE_TASK_LAST=undefined
SGE_TASK_FIRST=undefined
SGE_TASK_ID=undefined
[rp2927@researchgrid B9334]$
[rp2927@researchgrid B9334]$ cat bash-bat.e4949853
./HelloWorld.sh: line 6: mistaken_command: command not found
[rp2927@researchgrid B9334]$
```

```
[rp2927@researchgrid B9334]$
[rp2927@researchgrid B9334]$ cat HelloWorld.sh
#!/bin/bash
echo "Running on node: $(hostname)"
echo "Working directory: $(pwd)"
echo "Is there anybody out there?..."
echo -e "Here are the task control vars:\n$(env | grep SGE_TASK)"
mistaken_command
exit
[rp2927@researchgrid B9334]$
```

Batch Resource Limits

- Different behavior of interactive and batch execution modes with regard to momentary resource availability. If the requested resources are not available:
 - Interactive jobs will timeout after 5 sec, returning an error
 - Batch jobs will wait in queue (indefinitely)
- Regular user limits (under periodic admin adjustment)
 - 1000GB RAM
 - 500cores@regular ;; 100cores@longJobs
 - Runtime(wallclock): 480hr@regular ;; 100days@longJobs
- --grid_mem
 - Insufficient mem allocation will get your job killed!
 - Excessive mem allocation will waste the resource creating shortages for your colleagues
 - Increase the request in small increments. Default 4GB, although certain programs have a higher default
 - For multi-CPU jobs, the request above is the TOTAL mem allocation (not per CPU)
- --grid_ncpus
 - Ncpu is limited by the number of cores present on an actual worker node*
 - Using more threads than the number of allocated cores will decrease the performance.
**The number of cores per server varies.*

Deeper down the rabbit hole...

My job had disappeared with empty or absent logs, and no other trace...!

>>> Most probably the job was killed by the scheduler for insufficient memory allocation, but can we confirm?...

1. *(If you can, find the (7digit) job ID from the submit command reply, from an email notification or from the job's log files names – it is the number appended to the log file name following the type suffix {o, e, po, pe}, and use it below as argument to the “-j” option)*
2. Use “**qacct -o <UNI> -d <N> -j**” to display your recent accounting records.
 - *UNI* is your account, while the “-d <N>” option limits the search to the last N days.
 - Attention: *use a reasonable N interval, just long enough to capture the time of the event* and do not omit it, as the accounting log is huge going back a few years. Also, do not omit your UNI or you'll end up with everyone's records.
3. Use job parameters like qsub/start/end time and *jobname* to identify the record of the job you are interested in.
4. Look at the fields “failed” and “exit_status”:
 1. A job killed by the scheduler (for mem overreach) will have an **exit status** of **137** and **failed** code of **100**
 1. Failed code 100 = “Assumedly after job” (job ran, killed by signal)
 2. Exit code 137 = (128 + kill signal) => kill -9 (SIGKILL)
 2. A properly completed job will have both “failed” and “exit_status” codes equal to 0. (If the job terminates abnormally due to some program error, the exit status might be different (e.g. 1), but the “failed” code will remain 0.


```
rp2927 — ssh research.gsb.columbia.edu — 124x57
(live)rp2927@researchint01:/shared/share_B9334/Razvan$
(live)rp2927@researchint01:/shared/share_B9334/Razvan$ grid_run --grid_submit=batch --grid_mem=20G ./balloon.py
Your job 6586375 ("balloon.py") has been submitted
(live)rp2927@researchint01:/shared/share_B9334/Razvan$
(live)rp2927@researchint01:/shared/share_B9334/Razvan$ qstat
job-ID prior name user state submit/start at queue slots ja-task-ID
=====
6586375 0.55239 balloon.py rp2927 r 02/02/2022 14:39:38 debian.q@research124.grid.gsb 1
(live)rp2927@researchint01:/shared/share_B9334/Razvan$
(live)rp2927@researchint01:/shared/share_B9334/Razvan$ qstat
(live)rp2927@researchint01:/shared/share_B9334/Razvan$
(live)rp2927@researchint01:/shared/share_B9334/Razvan$ qacct -o rp2927 -d 1 -j 6586375
=====
qname      debian.q
hostname   research124.grid.gsb
group      rp2927
owner      rp2927
project    NONE
department defaultdepartment
jobname     balloon.py
jobnumber   6586375
taskid      undefined
account     sge
priority    0
qsub_time   Wed Feb 2 14:39:28 2022
start_time  Wed Feb 2 14:39:38 2022
end_time    Wed Feb 2 14:42:43 2022
granted_pe  threaded
slots       1
failed      0
exit_status 1
ru_wallclock 185
ru_etime    177.146
ru_stime    8.185
ru_maxrss   19627664
ru_ixrss    0
ru_ismrss   0
ru_idrss    0
ru_isrss    0
ru_minflt   3178145
ru_majflt   0
ru_nswap    0
ru_inblock  16
ru_oublock  168
ru_msgsnd   0
ru_msgrcv   0
ru_nsignals 0
ru_nvcsw    1111
ru_nivcsw    2698
cpu          185.331
mem          364.352
io           0.003
iow          0.000
maxvmem      3.696G
arid         undefined
(live)rp2927@researchint01:/shared/share_B9334/Razvan$
(live)rp2927@researchint01:/shared/share_B9334/Razvan$

jobname     python
jobnumber    6586362
taskid       undefined
account      sge
priority     0
qsub_time    Tue Feb 1 23:54:30 2022
start_time   Tue Feb 1 23:54:36 2022
end_time     Tue Feb 1 23:54:44 2022
granted_pe   threaded
slots        10
failed       100 : assumedly after job
exit_status   137
ru_wallclock 8
ru_etime     6.482
ru_stime     0.142
ru_maxrss    84940
ru_ixrss     0
ru_ismrss    0
ru_idrss     0
ru_isrss     0
ru_minflt    15300
ru_majflt    0
ru_nswap     0
ru_inblock   40
ru_oublock   192
ru_msgsnd    0
ru_msgrcv    0
ru_nsignals  0
ru_nvcsw     1876
ru_nivcsw     11
cpu          15.230
mem          3.104
io           0.009
iow          0.000
maxvmem      3.391G
arid         undefined
```

More about the accounting record...

- Jobname
- Timestamps
- No. slots
- (!) Exit codes (Failed, Exit_status)
- (!) Resource Usage:
 - ru_wallclock (sec)
 - **ru_maxrss** (max resident set size in kB)
- ** Ignore maxvmem (!) ☹

Selecting the right amount of memory

- Unfortunately, it is not a straightforward task...
- It depends on the number and (size) types of objects your code had created and handles at any given time.
- The size of your input data provides a hint, but what it's done with the inputs matters even more.
- It varies during the program's execution! (thus, the job might start and be running for awhile, but be killed later into the run)
- We need to size the job submit `grid_mem` option using the MAX value of the “virtual memory” (VmPeak)
- Given the complexity of estimating beforehand the VmPeak, we recommend a trial-and-error approach, followed if all possible by a downsizing based on post-run information.
 1. Start with an educated guess and increase the value of `grid_mem` option until the job runs successfully. Use reasonably small increments (10-20%?)
 2. Once the job ran successfully read the value of VmPeak from `/proc/$PPID/status` to find the maximum size of the virtual memory attained during the run and scale `grid_mem` down to it.

```
import os
os.system('grep VmPeak /proc/$PPID/status')
```
- Make all efforts to not waste system memory – it's a shared resource!



Batch Workflow: Large scale production



Loosely coupled parallel jobs

What if:

"I have hundreds of input files to be analyzed with the same algorithm"

"I need to run the same analysis with hundreds of parameter values"

"I need to run a distributed optimization scanning a parameter space with hundreds of points"

"I just sent out 100s of jobs and I discovered a mistake. How do I delete them?"



Array jobs: bundle a set of jobs (tasks) and treat it as a single processing job (create, delete, change, etc.)

SGE Array Jobs

- A set of tasks, handled unitarily, accessing an environment variable iterator
- Iterator takes integer values between two limits, with given step
- (can limit the number of concurrently running jobs)
- Create with GRID option:
 - `--grid_array=<first>-<last>[:<step>][/<conc>]`
 - `--grid_SGE_TASK_ID= =<first>-<last>`
- Three env variables are available to each task:
 - `SGE_TASK_ID` *Takes integer values from <first> to <last>, by step <step>*
 - `SGE_TASK_FIRST` *<first>*
 - `SGE_TASK_LAST` *<last>*

Array Jobs -- Example

Submit an array job using one version of the required Grid option.
Observe the job array index in the last column of qstat

```
[rp2927@researchgrid B9334]$  
[rp2927@researchgrid B9334]$ sge_run --grid_submit=batch --grid_SGE_TASK_ID=1-5 ./HelloWorld.sh  
Your job-array 4949855.1-5:1 ("bash-bat") has been submitted  
[rp2927@researchgrid B9334]$ qstat
```

job-ID	prior	name	user	state	submit/start at	queue	slots	ja-task-ID
4949855	0.50236	bash-bat	rp2927	r	02/07/2019 10:56:50	B9334@research47.gsb.columbia.	1	1
4949855	0.50118	bash-bat	rp2927	t	02/07/2019 10:56:51	B9334@research47.gsb.columbia.	1	2
4949855	0.50079	bash-bat	rp2927	t	02/07/2019 10:56:51	B9334@research47.gsb.columbia.	1	3
4949855	0.50059	bash-bat	rp2927	t	02/07/2019 10:56:51	B9334@research47.gsb.columbia.	1	4
4949855	0.50047	bash-bat	rp2927	t	02/07/2019 10:56:51	B9334@research47.gsb.columbia.	1	5

```
[rp2927@researchgrid B9334]$ qstat  
[rp2927@researchgrid B9334]$  
[rp2927@researchgrid B9334]$
```


Array Jobs -- Logs

The array jobs will generate one set of 4 logs per task

```
[rp2927@researchgrid B9334]$ ls -lrt
total 176
-rwxrwxr-x 1 rp2927 578103723 191 Feb  7 10:51 HelloWorld.sh
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.po4949855.1
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.pe4949855.1
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.po4949855.2
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.pe4949855.2
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.po4949855.3
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.pe4949855.3
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.e4949855.1
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.po4949855.4
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.pe4949855.4
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.e4949855.2
-rw-r--r-- 1 rp2927 578103723 202 Feb  7 10:53 bash-bat.o4949855.1
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.po4949855.5
-rw-r--r-- 1 rp2927 578103723 202 Feb  7 10:53 bash-bat.o4949855.2
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.pe4949855.5
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.e4949855.3
-rw-r--r-- 1 rp2927 578103723 202 Feb  7 10:53 bash-bat.o4949855.3
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.e4949855.4
-rw-r--r-- 1 rp2927 578103723 202 Feb  7 10:53 bash-bat.o4949855.4
-rw-r--r-- 1 rp2927 578103723   0 Feb  7 10:53 bash-bat.e4949855.5
-rw-r--r-- 1 rp2927 578103723 202 Feb  7 10:53 bash-bat.o4949855.5
[rp2927@researchgrid B9334]$
```

Task index

```
[rp2927@researchgrid B9334]$
[rp2927@researchgrid B9334]$ cat bash-bat.o4949855.2
Running on node: research47
Working directory: /user/user1/rp2927/B9334
Is there anybody out there?...
Here are the task control vars:
SGE_TASK_STEPSIZE=1
SGE_TASK_LAST=5
SGE_TASK_FIRST=1
SGE_TASK_ID=2
[rp2927@researchgrid B9334]$
[rp2927@researchgrid B9334]$
```

SGE Array Jobs – Timing and synchronization

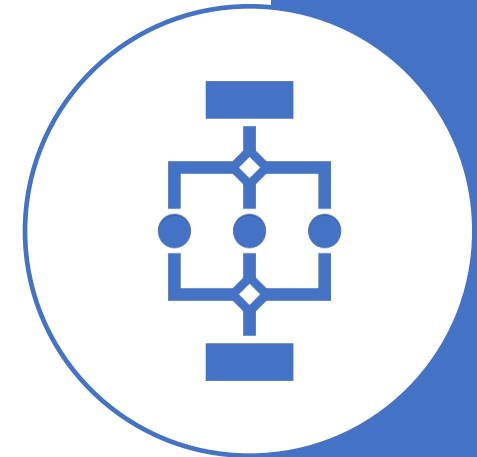
The array tasks will be started in sequence: <first>/1, 2, 3 ... <last>

However, task completion time will depend not only on the computational effort required by each parameter set, but also on the machine speed and the average load of the worker where they will be instantiated. (True for both heterogeneous and homogeneous clusters)

Consequently, the task termination time is unpredictable, and the array tasks completion times will be **out of sequence!**

SGE Job Synchronization

- Schedule a parallel computation workflow composed of:
 1. A preparer job (e.g. split input data in equally sized sub-inputs)
 2. An array job (main workload: process the sub-inputs in parallel)
 3. A finisher job (e.g. summarize output of individual tasks from the array job)
- It is essential that each step does not start until the previous one completes!
 - Use GRID “hold” option: “**--grid_hold=<jobID>**”, where <jobID> is the jobId of the **precursor** job.
 - **grid_run** <gridOptions> <scriptName> <scriptParams>
 - E.g. **grid_run** --grid_submit=batch --grid_mem=25G --grid_hold=615237 ./myScript.sh





Python
(Anaconda)

Development with Python Code

Caution!

- Several Python interpreters are installed.
 - The **system default** (/usr/bin/python) – used by the OS. Older version without data science packages. **DO NOT USE!**
 - The Anaconda system installation (/apps/anaconda3/bin/python). Great choice!
 - Private environment. Most flexible choice! (and most complex)



- Preconfigured commands invoking Anaconda Python
 - anapy3
 - ipython3
 - spyder
 - notebook
 - jupyterlab

Obs.: All the commands above accept GRID options. E.g.:

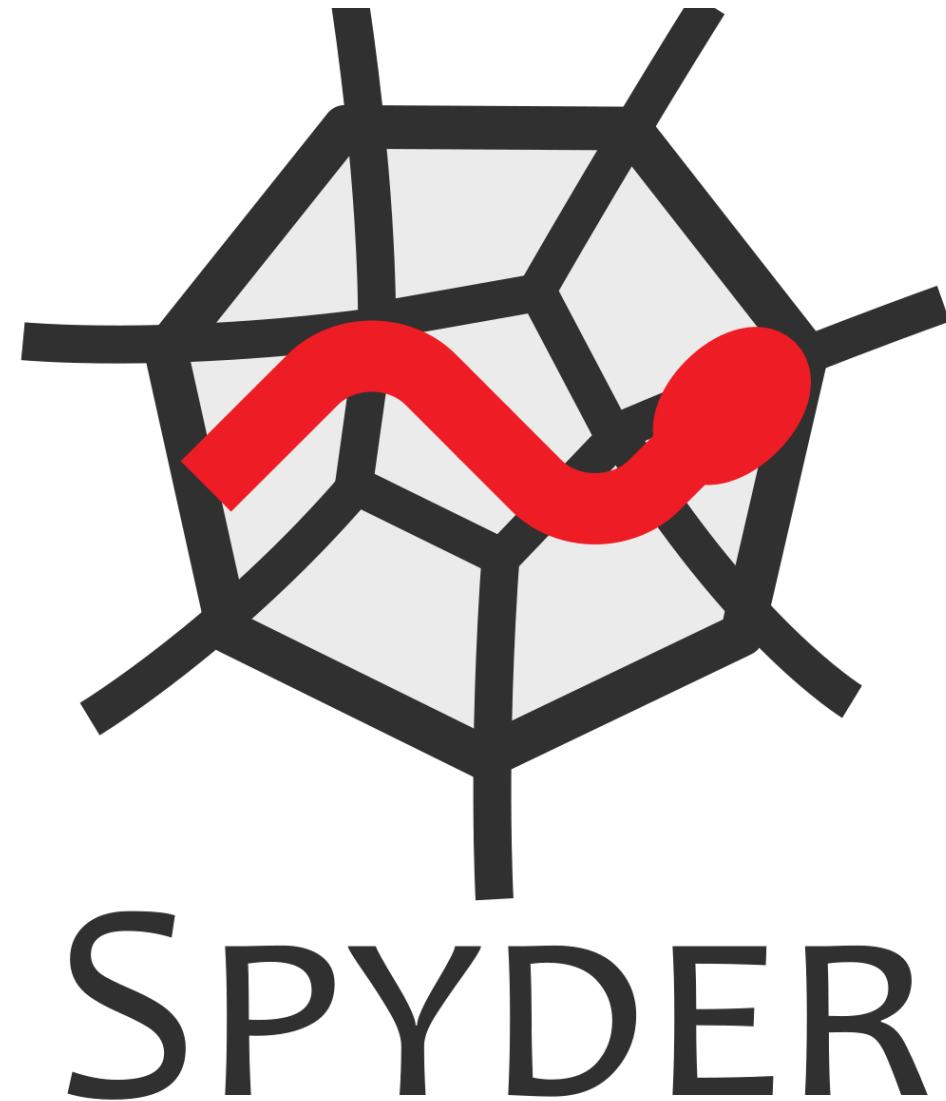
- *anapy3 --grid_mem=50G*
- The grid_run general handler using a script that includes a (shebang) properly defined interpreter

Python Interpreters and IDEs

- Command line interpreter:
 - `anapy3`
- IPython:
 - `ipython3`
- Spyder (requires graphic support):
 - `spyder`
- VS Code (client/server model)

Obs.: All the commands above accept GRID options. E.g.:

- `spyder --grid_mem=50G`



Jupyter Notebook / Lab

(https://wiki.gsb.columbia.edu/research/index.php/Jupyter_Notebook/Lab)

- Project Jupyter Notebook and Lab interfaces are available on the CBS cluster, using a **server/client model** where the computation is performed on a Grid node and the presentation is done by a web browser running on the local system. Consequently, the startup requires two steps.
 1. Launch, typing “**notebook**” or “**jupyterlab**”
 2. Connect the **local web browser** to the Jupyter server:
 - Copy the **URL containing “gsb.columbia.edu”** displayed at step 1, and paste it into a **local web browser**.
 - DO NOT USE Firefox from the virtual desktop!!

Obs.: All the commands above accept GRID options. E.g.:

- `notebook --grid_mem=50G`



JupyterHub



- Web service that provides individual **JupyterLab servers** running on the GRID cluster worker nodes. Customized locally to allow selecting the CPU core count and the amount of RAM.
 - Still having some issues! (*delayed selection of resources*)
- Available at: <https://research.gsb.columbia.edu:8000/>
- No need to login via ssh or NoMachine.
- *Persistent* servers
- Requests are processed through the GRID scheduler, so all CPU/MEM/time limitations apply.

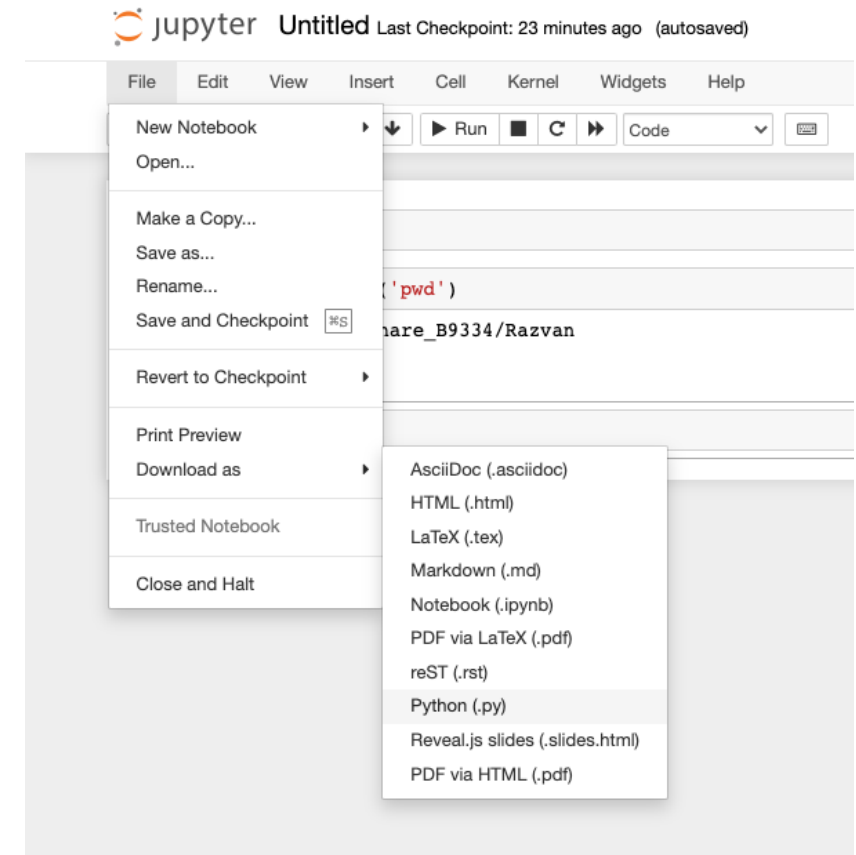
From Notebook development to production

Develop the prototype of a new analysis in a Jupyter Notebook and then run the result as a production (batch) job.

1. Develop the code within a Jupyter notebook using the launch method mentioned before (“notebook...grid_options...”). Request an appropriate amount of resources (CPU, RAM) using GRID options on the “notebook” command.
2. Export the notebook as a Python (.py) script (Menu: File > Download as > Python (.py))
3. Transfer the saved script to the GRID
4. Edit the script's shebang (!) changing:
 from “#!/usr/bin/env python”
 to “#!/apps/bin/anaconda3/bin/python”
5. Run/submit the script using *grid_run* (--grid_submit=batch)

Method 2 (simpler conversion):

1. Run “ipython3 nbconvert --to python <notebookName>”
2. Perform steps 4 and 5 as above.



Running Python code (non-interactive/batch mode)

- Prefix the file containing the Python code with the (shebang) line corresponding to **the Python interpreter you want to use**:

- `#!/apps/anaconda3/bin/python` *(avoid #!/bin/env python)*

- Run with “grid_run”:

- `grid_run --grid_mem=20G ./myScript.py`

- *Would this work?*

- `anapy3 --grid_mem=20G ./myScript.py`

- *What is the difference between the two invocations?*





Python modules and environments



Python distributions (Anaconda)

- Baseline: Anaconda Python
 - Distribution of Python optimized for scientific computing
 - Python interpreter
 - Set of (~350) packages focused on data science
 - Package manager (conda)
 - Centrally installed on the GRID@CBS cluster
 - *Unique copy* maintained by ResSupport -- *upgraded periodically*
 - Follows the tested releases by Anaconda (bi-annually)
 - Changes (bi-annually)
 - Is restricted to the content of the tested bundles (meta-packages) released by Anaconda

Caution: Variability! The list of installed packages (and their corresponding versions) will change. Otherwise stable code might suddenly stop working.

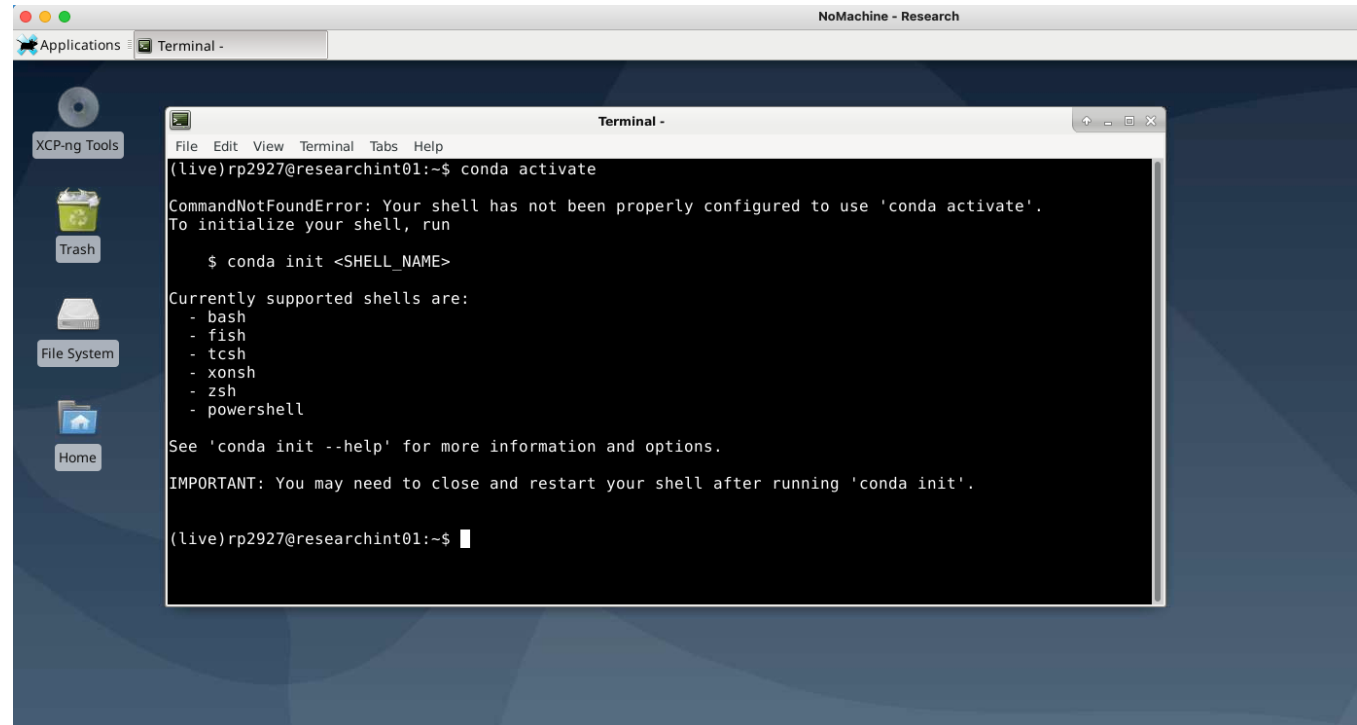
Conda – Anaconda package manager



- Versatile package manager created by the Anaconda project
 - Used to manage Python packages and environments
 - Install Python packages, managing channels, resolving dependencies and conflicts
 - `conda install <packageName>[=<version>]`
 - Create and manage private environments
 - `conda create -n <envName> <packageName>[=<version>]`
 - E.g. `conda create -n myNewEnv python=3.9 keras`
 - Activate/deactivate Python environments
 - `conda activate <envName>`
 - E.g. `conda activate myNewEnv`
 - E.g. `conda activate` Activates the “base” environment (the system Anaconda install)
 - Inspect state
 - `conda info` Displays current state (configuration) – includes current env.
 - `conda list` Displays the list of packages from the current environment

Anaconda activation and the Nomachine virtual desktop

Anaconda (manual) activation (“conda activate”) *within the virtual desktop* fails and requires to setup automatic activation (“conda init”).



The screenshot shows a virtual desktop environment titled "NoMachine - Research". On the left sidebar, there are icons for "XCP-ng Tools", "Trash", "File System", and "Home". A terminal window is open in the center, displaying the following text:

```
(live)rp2927@researchint01:~$ conda activate
CommandNotFoundError: Your shell has not been properly configured to use 'conda activate'.
To initialize your shell, run

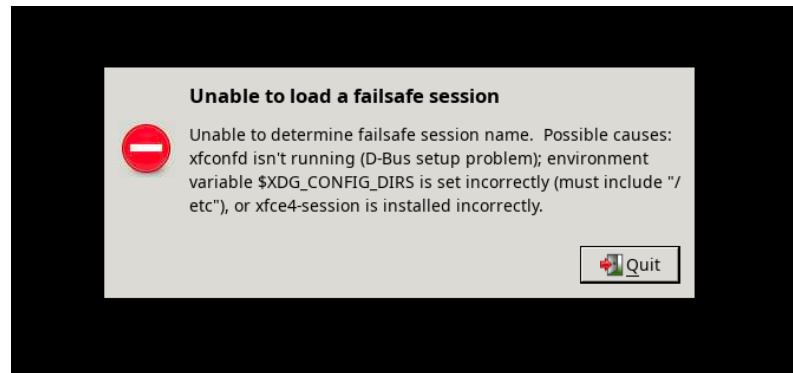
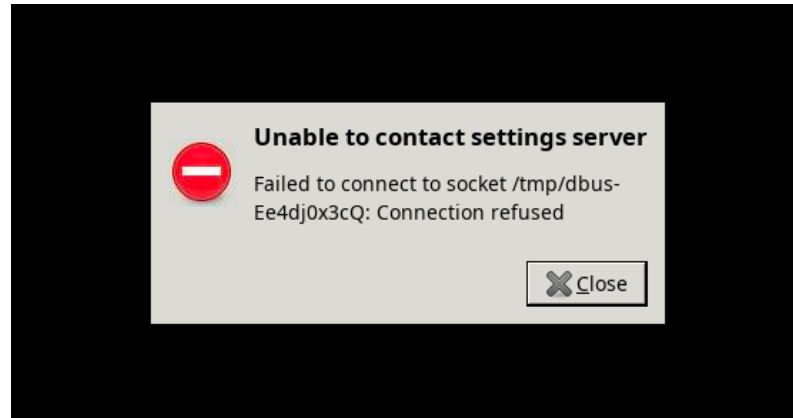
    $ conda init <SHELL_NAME>

Currently supported shells are:
- bash
- fish
- tcsh
- xonsh
- zsh
- powershell

See 'conda init --help' for more information and options.
IMPORTANT: You may need to close and restart your shell after running 'conda init'.

(live)rp2927@researchint01:~$
```

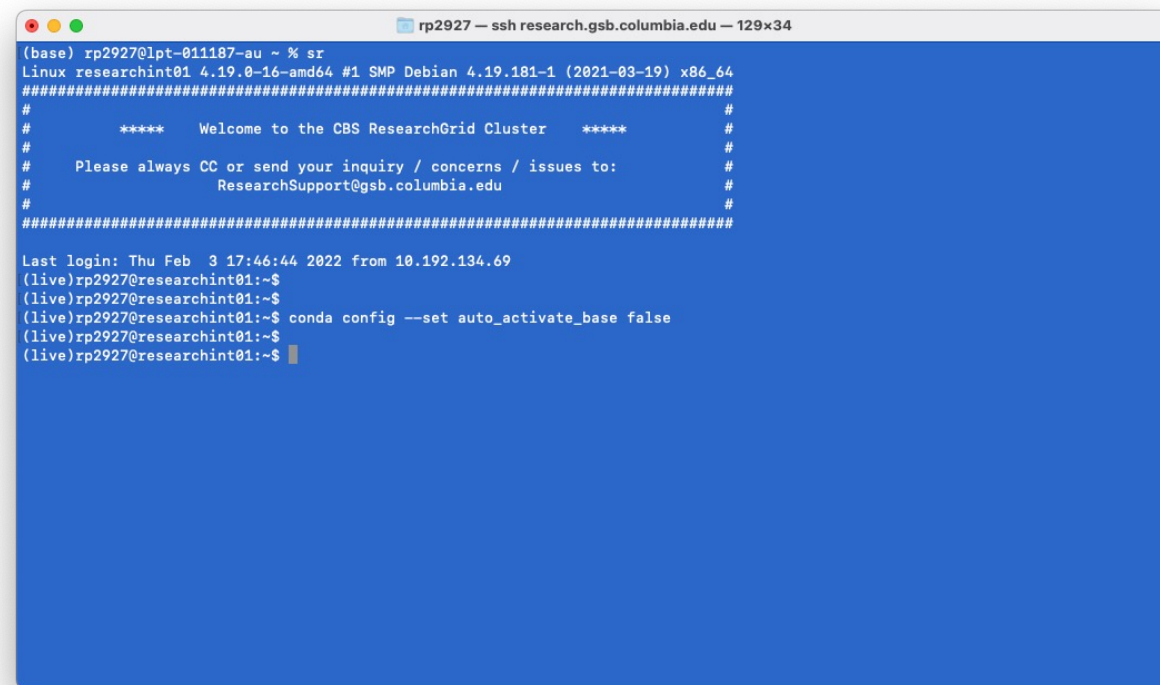
Anaconda activation and the Nomachine virtual desktop



- Unfortunately, Anaconda automatic activation breaks the Nomachine virtual desktop.

Anaconda activation and the Nomachine virtual desktop

- Compromise: let conda setup the shell but disable automatic base environment activation
 - Use a floating terminal or ssh via PuTTY or MacTerminal!...
 - **conda config --set auto_activate_base false**
- *Preferably: perform this additional step immediately after running “conda init”, to avoid being locked-out from the virtual desktop in the first place!*
- Will need to activate the base env manually, when needed!



```
rp2927 — ssh research.gsb.columbia.edu — 129x34
(base) rp2927@lpt-011187-au ~ % sr
Linux researchint01 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64
#####
#
#      *****  Welcome to the CBS ResearchGrid Cluster  *****
#
#      Please always CC or send your inquiry / concerns / issues to:
#                      ResearchSupport@gsb.columbia.edu
#
#####

Last login: Thu Feb  3 17:46:44 2022 from 10.192.134.69
(live)rp2927@researchint01:~$
(live)rp2927@researchint01:~$
(live)rp2927@researchint01:~$ conda config --set auto_activate_base false
(live)rp2927@researchint01:~$
(live)rp2927@researchint01:~$
```

Python workflow problems (missing modules)

```
conda activate  
pip install <pckgName>[==version]
```

- A. Start coding using the system provided Anaconda distribution
- Use GRID predefined commands to invoke the interpreter/IDE interactively ([anapy3](#), [spyder](#), [notebook](#), [VS code](#))...
 - Or run your code unattended using [grid_run](#) and scripts that employ the shebang:
 - `#!/apps/anaconda3/bin/python`
- B. Problem 1: **An import statement fails**; I need a module that is not available. How do I add a Python package that is not provided by the system Anaconda?
- 1) Use the *conda* package manager to install it?!?
 - **No, conda won't be able to write in the (protected) system Anaconda location! An explicit error will be thrown.**
 - 2) We need a different installer that can be steered to install in a personal location: *pip*
 - 3) SOLUTION:
 - i. First, run “*conda activate*” to activate the base (system) Anaconda and establish the context that *pip* will use to resolve dependencies and conflicts. (Also, this step will make the *pip* command available)
 - ii. Then, install the required packages: “*pip install <packageName>[==<version>]*”

Obs.1: By default, pip will install new packages in `~/local/lib/pythonX.X/site-packages` which is a location part of the default Python search path (`sys.path`). Therefore no special action is required to help Python find the new additions. However, if this fails, or if you install in a different location, use the `PYTHONPATH` variable to include the installation location in the Python search path (add to `~/bashrc`):

- `export PYTHONPATH=/your/package/installation/location`

Obs.2: The local package library is Python version specific, therefore when a new interpreter version is deployed all local packages need to be reinstalled.

Python virtual environments



- Python (virtual) environment: a self-contained **context** providing a working software infrastructure that includes interpreter, libraries, scripts and tools, in support of Python software development.
- Purpose?!
 - Provide **multiple, self-contained, stable** Python development contexts
 - Reliability: remove dependency on shared elements that can change outside user control
 - Maintainability: select compatible, known-to-work, infrastructure components (packages) and supply a reliable method to control them, including the ability to replicate contexts (portability!)
- Manage via:
 - Conda {create, install/upgrade, activate/deactivate}
 - (... Implemented using env variables (PATH))

Python workflow (module incompatibilities)

Problem 2: *“Everything was working fine before, but now my code is crashing!”*

- Changes made to the system Anaconda packages won't update your personal repos!
 - Unidirectional dependencies. Disjoint validation domains.
 - Potential for breaking user code! (introducing regression)

Problem 2': *“I need to install a module that is available only as a conda package”*

- Cannot add to the base environment using “conda install” (not writable).

1) Must decouple from the system Anaconda. **Create a private virtual environment.**

- `conda create -n <myNewEnv> [<packageName>[=<version>]]`

2) SOLUTION:

- First, create a new environment using `“conda create -n <myNewEnv> [<packageName>[=<version>]]”`
- Next, activate the new environment: `“conda activate <myNewEnv>”`
- Then, install all the required packages: `“conda install <packageName>[=<version>]”`

Python workflow – working with private envs

If a private virtual environment is used, the guidance given so far for working with Anaconda Python must be adapted!

A. Running interactively:

- The preconfigured commands {*anapy3*, *ipython3*, *spyder*, *notebook*, etc.} will not work!
- *?... conda activate <myNewEnv>*
- *?... [i]python*
 - *Caution: the command above starts the interpreter locally, on the interactive node. Do not use for heavy computation, to avoid disrupting your colleagues.*
- *!... grid_run can help but must be adapted:*
 - *E.g.: grid_run <grid_options> /user/UNI/.conda/envs/myNewEnv/bin/spyder*
 - *E.g.: grid_run <grid_options> /user/UNI/.conda/envs/myNewEnv/bin/jupyter-notebook --ip=\$(hostname).gsb.columbia.edu*

B. Running code:

- Replace the shebang line with the location of the python interpreter from your private environment and run with *grid_run*:
 - E.g.: *#!/user/<myUNI>/.conda/envs/<myNewEnv>/bin/python*



Working with private environments -- IDE

- Notebook/JupyterLab:
 - Can activate a different kernel (installed in your private env) if you make sure you add the “**ipykernel**” module to the private env (conda install ipykernel)
 - Start the (GRID) notebook/jupyterLab with the usual command and then switch kernel.
- JupyterHub:
 - Same procedure as above – make sure you added *ipykernel* to the env and switch kernel from the interface.
- VS Code:
 - To use the new private env select the respective interpreter from the command palette (Python: Select Interpreter) or by clicking on the lower right corner.

Private env – TF on GRID_GPU

A typical example for needing a private env. @ GRID is the use of the GPU enabled servers. The local deployment model requires that one installs CUDA toolkit from conda, thus conflicting with use of the base environment.

- Requires interactive access to the GPU enabled nodes
- The installation of TF follows the recommendations from <https://www.tensorflow.org/install/pip>

1. `grid_run --grid_gpu`
2. `conda create --name tf python=3.9`
3. `conda deactivate`
`conda activate tf`
4. `conda install -c conda-forge cudatoolkit=11.2 cudnn=8.1.0`
5. `export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$CONDA_PREFIX/lib/`
6. `mkdir -p $CONDA_PREFIX/etc/conda/activate.d`
`echo 'export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$CONDA_PREFIX/lib/' > $CONDA_PREFIX/etc/conda/activate.d/env_vars.sh`
7. `pip install tensorflow`
8. `python3 -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"`

Use the correct shebang in the new scripts:

```
#!/user/UNI/.conda/envs/tf/bin/python
```

Start as usual but include the GPU selector GRID option:

```
grid_run --grid_submit=batch --grid_gpu ./myScript.py
```


A microscopic view of a multi-processor chip, showing a grid of processing units and several parallel lanes of circuitry in various colors (pink, yellow, green, blue) running diagonally across the frame.

Process-based parallelism

Python multiprocessing

Threads, subprocesses and GIL

- Processes and threads
 - Process: Operating System set of structures controlling one activity (element of computational work)
 - Code
 - Data structures (stack, heap, registers) – objects created by the code, and state info
 - Threads: Elements of execution within one process (enable the use of multiple CPUs)
 - Share memory space!
 - Are “easy” to create and destroy
 - Subprocesses:
 - Processes created by a “parent” process. Have all attributes of a regular process.
- GIL – the Python “Global Interpreter Lock”
 - One per interpreter process (CPython)
 - Synchronizes the execution of threads such that only one native thread can execute at a time!
- Best process-based parallelism implementation will depend on the nature of the job!

Types of process-based parallelism

- **Multiprocessing**

- Uses multiple (sub)processes to concurrently execute work dispatched by the main process (multiprocess pool).
- Best adapted for CPU bound workloads.
- Each subprocess has its own GIL and can execute in parallel, making use of multiple CPU cores.
- Separate memory spaces. A bit more costly to create and destroy.

- **Threading**

- Uses multiple threads in the same process.
- Subject to GIL restrictions (in CPython)
- Best adapted for I/O bound workloads
- Shares memory space. Simpler coordination and smaller memory footprint.

- **Asyncio**

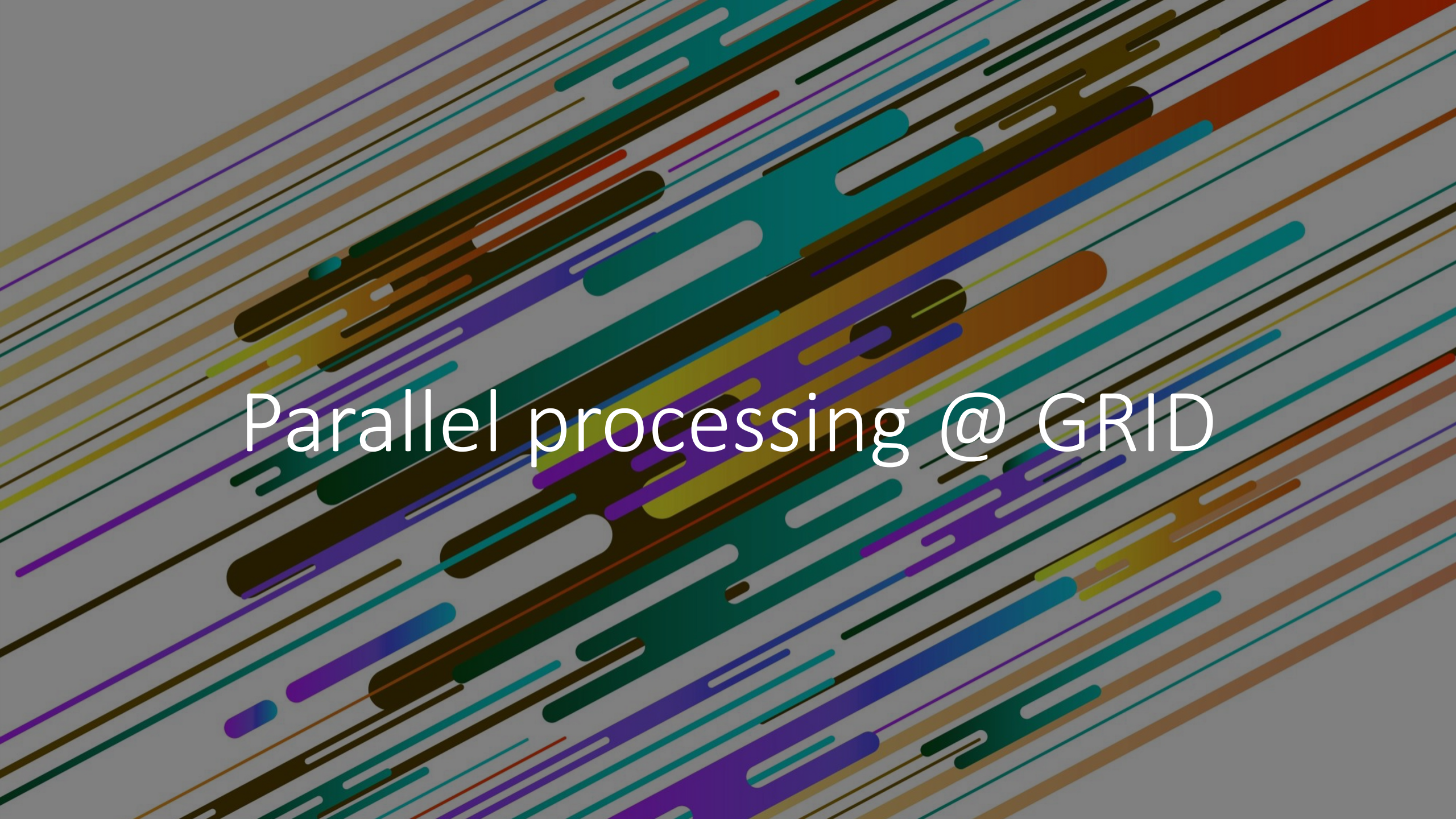
- Best adapted for (slower) I/O loads
- Cooperative multitasking

Using the “multiprocessing” module

- By default, it allows to run a function via a separate sub-process.
 - Needs explicit creation, start and sync (join) of worker process: using *Process()*, *start()* and *join()* calls.
- Parent and worker memory spaces are separate.
 - Need a mean to transfer data!
- Sharing data between processes:
 - Shared memory: *Array* and *Value* (shared) objects.
- Inter-process communication:
 - *Queue* and *Pipe* objects.
- Managing workers:
 - *Pool* class creates a set of worker subprocesses
 - The *map()* or *starmap()* calls dispatches work to the workers (data parallelism)

Python data parallelism with multiprocessing

```
1  #!/apps/anaconda3/bin/python
2
3  # Example of data parallelism using multiprocessing Pool and Queue
4
5  ∨ import multiprocessing
6  import os
7
8  # Define the payload function
9  ∨ def func(x, q):
10     |     q.put(x * x)
11
12  ∨ def main():
13
14     # Get the number of allocated cores
15     cores = int(os.getenv('NSLOTS'))
16
17     # Create a multiprocessing pool of size equal to the number of allocated cores
18     pool = multiprocessing.Pool(cores)
19
20     # Create a manager
21     m = multiprocessing.Manager()
22
23     # Create a queue
24     q = m.Queue()
25
26     # Map the payload function onto a set of arguments
27     pool.starmap(func, [(a, q) for a in range(10)])
28
29     # Retrieve the messages from the queue
30  ∨ while not q.empty():
31     |     print(q.get())
32
33     # Close the pool
34     pool.close()
35
36  ∨ if __name__ == '__main__':
37     |     main()
38
```

Parallel processing @ GRID

Bringing it all together

- A fully automated parallel processing workflow using all GRID resources and features:
 - Bash main control script that launches the complete workflow as a job-array with synchronized preparation and summarization steps.
 - Collects input from user and determines how to configure the parallel processing.
 - Submits data preparation (splitting) batch job and retrieves jobID
 - Submits a batch array-job for parallel processing of sub-inputs placing a hold for the previous step. Retrieves the jobID of the array job.
 - Submits a final merger batch job placing a hold for the previous step.
 - The programs at each step (preparation, processing, merger) use multiprocessing to leverage multiple CPU cores available on the cluster.



Running R

- Same as with Anaconda Python, there is a (shared) system R installation. The GUI frontend “**Rstudio**” is also available.
- Invoke by simply typing: “**R**” or “**Rstudio**”
- Execute (batch) R code using “**Rscript**”:
 - **Rscript** [<gridOptions>] <scriptName> <scriptParams>
- ... or using **grid_run** and the **#!/apps/bin/Rscript** shebang

Obs.: All the commands above accept GRID options. E.g.:

- **R --grid_mem=50G**



Installing (user) R packages

[https://wiki.gsb.columbia.edu/research/index.php/R_\(new\)](https://wiki.gsb.columbia.edu/research/index.php/R_(new))

- As the system R installation cannot be altered by users, adding new R packages must be done with the help of a user library.
- The software will automatically detect your intention and prompt you to create a personal lib at the proper location, if it doesn't already exist.

Installing R packages in user libraries [\[edit\]](#)

It is frequently much more convenient to install R packages in a personal library rather than ask the system administrators for a system-wide installation. In addition, a system installation will certainly change, updating various packages based on requests from other members of the community, while a personal/user library will always be in a known state. The downside of the latter is that a certain amount of maintenance is required.

In order to find packages, R looks at the variables `R_LIBS_USER` and `R_LIBS_SITE`. `R_LIBS_USER` is by default set to:

```
R_LIBS_USER=${R_LIBS_USER:- '~/x86_64-pc-linux-gnu-library/3.6'}
```

Therefore the default location of a user library is:

```
/user/<userName>/R/x86_64-pc-linux-gnu-library/3.6
```

You can always check the library search list, within an R session, using:

```
> .libPaths()
```

A typical answer would be:

```
> .libPaths()  
[1] "/user/<yourUserNameHere>/R/x86_64-pc-linux-gnu-library/3.6"  
[2] "/apps/R-3.6.0/lib64/R/library"
```

If you've never installed any local package the first path won't be listed, as it was never created. So let's install a package in the user library.

Simply open an R session and type:

```
> install.packages("packageName")
```

As you don't have write access to the system library, you'll be asked whether you want to install it in a personal library.

```
> install.packages("packageName")  
Warning in install.packages("packageName") :  
  'lib = "/apps/R-3.6.0/lib64/R/library"' is not writable  
Would you like to use a personal library instead? (yes/No/cancel) y  
Would you like to create a personal library  
  '~/x86_64-pc-linux-gnu-library/3.6'  
to install packages into? (yes/No/cancel) y  
--- Please select a CRAN mirror for use in this session ---  
Secure CRAN mirrors  
[...]
```

That's it. Next time you'll need to install other packages the initial dialog won't exist as the system will detect your personal library and will use that default target:

```
> install.packages("stringi")  
Installing package into '/user/<userName>/R/x86_64-pc-linux-gnu-library/3.6'  
(as 'lib' is unspecified)  
trying URL 'https://cloud.r-project.org/src/contrib/...[...]
```

Obs. Keep in mind that the packages will be built for the R version you're running when installing them (here R 3.6 from the new cluster). If you're using multiple R versions certain packages, built for other versions, might fail. To avoid that outcome the personal library default location is version specific (observe the "3.6" in its path) and therefore multiple libraries will have to be maintained.



Support

researchsupport@gsb.columbia.edu